دانشگاه صنعتی قوچان

گروه مهندسی برق

پایان‌نامه دوره‌ی کارشناسی مهندسی برق–گرایش الکترونیک

پنل نمایشگر عکس با قابلیت کنترل از راه دور

محمد هادی نجفی

استاد راهنما:

جناب دکتر مرتضی رجب زاده

زمستان 1399

بسم الله الرحمن الرحیم

تقدیم

به پدر و مادرعزیزم که همواره پشتیبان و بزرگترین سرمایه در زندگی برای من بودند. همچنین جا دارد از زحمات جناب مهندس مجید توکلی که در دوران نوجوانی مربی و مشوق من در راه پیشرفت بودند تشکر کنم.

تأییدیه هیئت داوران

1- استاد راهنمای اول:

نام و نام خانوادگی: ---------- امضا: ----------

2- استاد راهنمای دوم:

نام و نام خانوادگی: ---------- امضا: ----------

3- استاد دفاع اول:

نام و نام خانوادگی: ---------- امضا: ----------

4- استاد دفاع دوم:

نام و نام خانوادگی: ---------- امضا: ----------

5- نماینده گروه مهندسی برق

نام و نام خانوادگی: ---------- امضا: ----------

اظهار نامه

اینجانب **محمد هادی نجفی** دانشجوی کارشناسی رشته مهندسی برق گرایش الکترونیک دانشکده مهندسی برق و کامپیوتر دانشگاه صنعتی قوچان، گواهی می‌دهم که پایان‌نامه تهیه شده توسط اینجانب با عنوان" **پنل نمایشگر عکس با قابلیت کنترل از راه دور**" به راهنمایی استاد محترم جناب آقای **دکتر مرتضی رجب زاده** توسط شخص اینجانب انجام گرفته و صحت و اصالت مطالب تدوین شده در آن را تأیید می‌نمایم. چنان‌چه هر زمان، دانشگاه آگاه شود که گزارش پایان نامه/رساله حاضر، صحت و اصالت لازم را نداشته است، آن دانشگاه حق دارد، مدرک تحصیلی اینجانب را مسترد و ابطال نماید. همچنین اعلام می‌دارد در صورت بهره گیری از منابع مختلف شامل، گزارش‌های تحقیقاتی، رساله، پایان‌نامه، کتاب، مقالات تخصصی و غیره، به منبع مورد استفاده و پدیدآورنده آن به طور دقیق ارجاع داده شده و نیز مطالب مندرج در پایان‌نامه/رساله حاضر تاکنون برای دریافت هیچ نوع مدرک یا امتیازی توسط اینجانب و یا سایر افراد به هیچ کجا ارائه نشده است. در تدوین متن پایان‌نامه/رساله حاضر، چارچوب (قالب) مصوب گروه مهندسی برق دانشگاه صنعتی قوچان (واقع در بخش دریافت فایل در تارنمای گروه مهندسی برق دانشگاه) به طور کامل رعایت شده است. در پایان اعلام می‌دارم کلیه حقوق مادی ناشی از پایان نامه/رساله حاضر، متعلق به دانشگاه مهندسی فناوری‌های نوین قوچان می‌باشد.

نام و نام خانوادگی دانشجو (دست نویس) .....

امضاء دانشجو:

تاریخ:

حق چاپ و تکثیر و مالکیت (Copyright and Ownership**) این پایان‌نامه**

به دانشگاه صنعتی قوچان واگذار شده است.

تشکر و قدردانی

گزارشی که در پیش رو دارید ، حاصل تلاش های چندین ماهه برای طراحی و ساخت یک پنل نمایشگر است. در این پروژه سعی شده از تکنیک های حرفه ای حوزه الکترونیک برای بهبود کارایی و کمتر شدن قیمت تمام شده پروژه استفاده شود.

جا دارد از راهنمایی های استاد محترم ، جناب آقای دکتر مرتضی رجب زاده تشکر کنم.

محمد هادی نجفی

دانشجوی مهندسی برق- الکترونیک

دانشگاه صنعتی قوچان

بهمن ماه 1399.

چکیده

با پیشرفت علم و ظهور نمایشگر [1]های LCD ، کاربرد های متنوعی برای گونه از این نمایشگرها در حوزه های مختلف تعریف شده است. هم اکنون ، شرکت های طراح و تولید کننده میکروکنترلر[2] ها، یک بخش سخت افزاری مجزا برای راه اندازی و نمایش محتوای تصویری روی این گونه نمایشگر ها، روی تراشه [3]های خود قرار می دهند که به اختصار LTDC نامیده می شوند. در این پروژه سعی شده تا باتوسعه نرم افزاری میکروکنترلر های فاقد LTDC ، این مکان را روی آن ها پیاده سازی کرد .

واژگان کلیدی: نمایشگر های LCD ، میکروکنترلرها ،تراشه ها ، LTDC

[1] Display
[2] Microcontroller
[3] Chipset

فهرست مطالب

عنوان **صفحه**

فهرست علایم و نشانه‌ها ..... ج
فهرست شکل‌ها ..... د

فصل 1– کلیات پروژه 6
1-1- مقدمه : 6
1-2- بیان مسئله پروژه :..... 6
1-3- اهداف پروژه :..... 6

فصل 2– سخت افزار 7
2-1- مقدمه : 7
2-2- نمایشگر TFT : 7
2-2-1- ارتباط با نمایشگر TFT :..... 8
2-2-2- نور پس زمینه در نمایشگر TFT : 10
2-3- PCB پروژه :..... 11
2-3-1- بخش تغذیه : 12
2-3-2- مدار حفاظت ورودی تغذیه :..... 12
2-3-3- رگولاتور سوئیچینگ 5 ولت : 14
2-3-4- رگولاتور های خطی 3.3 ولت : 15
2-3-5- رگولاتور سوئیچینگ 19 ولت : 16
2-3-6- پردازنده اصلی :..... 17
2-3-7- آی سی RAM :..... 19
2-3-8- درگاه کارت حافظه:..... 21
2-3-9- درگاه نمایشگر : 22
2-3-10- ماژول گیرنده مادون قرمز : 23
2-4- نتیجه گیری: 24

فصل 3- نرم افزار 25
3-1- مقدمه : 25
3-2- معرفی نرم افزار STM32CubeIDE : 25
3-3- ایجاد پروژه جدید در STM32CubeIDE : 25
3-4- تشریح کد های برنامه : 28
3-4-1- تابع اصلی برنامه (main) : 28

3-4-2- تابع SRAM_Precharge : 32
3-4-3- تابع SRAM_AutoRefresh : 32
3-4-4- تابع SRAM_RowBankActive : 33
3-4-5- تابع SRAM_Write : 33
3-4-6- تابع SRAM_RegisterSet : 34
3-4-7- تابع SRAM_BurstStop : 34
3-4-8- تابع SRAM_Init : 35
3-4-9- تابع PicToRam : 35
3-4-10- تابع LCD_Display : 36
3-4-11- تابع LCD_Init : 36
3-4-12- تابع LCD_Hsync : 36
3-4-13- تابع LCD_Show : 38
3-4-14- تابع Slide_Show : 38
3-4-15- تابع SystemClock_Config : 39
3-4-16- تابع MX_SDIO_SD_Init : 40
3-4-17- تابع MX_USART1_UART_Init : 41
3-4-18- تابع MX_GPIO_Init : 41
3-5- نتیجه گیری: 43

فصل 4- عملکرد دستگاه: 44
4-1- مقدمه: 44
4-2- نحوه عملکرد دستگاه: 44
4-3- نحوه انتقال عکس به کارت حافظه : 44
4-4- نتیجه گیری 45

فصل 5– نتیجه‌گیری و پیشنهادها 46
5-1- نتیجه‌گیری 46
5-2- پیشنهادها 46

ضمیمه أ – عکس هایی از پروژه 47

فهرست مراجع: 49

واژه نامه‌ی فارسی به انگلیسی 50

واژه نامه‌ی انگلیسی به فارسی 51

فهرست علایم و نشانه‌ها

عنوان	علامت اختصاری
هرتز	Hz
آمپر	A
ولت	V
بایت	Byte
بیت	Bit

فهرست شکل‌ها

عنوان **صفحه**

شکل 1-2: بخش های داخلی یک TFT-LCD. 6
شکل 2-2: نمایشگر 5 اینچ به همراه کابل فلت 40 پایه 7
شکل 3-2: تشریح پایه های کابل فلت 40 پایه 7
شکل 4-2: نحوه مقدار دهی سطر به سطر صفحه نمایش 8
شکل 5-2: نحوه مقدار دهی پیکسل های یک سطر از صفحه نمایش 8
شکل 6-2: شماتیک و مشخصات فنی بخش نور پس زمینه 9
شکل 7-2: یک نمونه LED نور پس زمینه در حالت روشن 9
شکل 8-2: PCB پروژه (رو). 10
شکل 9-2: PCB پروژه (پشت). 10
شکل 10-2: شماتیک کلی بخش تغذیه 11
شکل 11-2: شماتیک بخش ورودی تغذیه 12
شکل 12-2: دیود 1N5822 - SMD 12
شکل 13-2: فیوز 1 آمپر SMD. 12
شکل 14-2: شماتیک مدار رگولاتور 5 ولت. 13
شکل 15-2: رگولاتور LM2596 درکنار دیگر قطعات مورد نیاز آن. 13
شکل 16-2: شماتیک مدار رگولاتور های خطی AMS1117. 14
شکل 17-2: رگولاتور خطی AMS1117 به همراه خازن هاv. 14
شکل 18-2: شماتیک مدار رگولاتور سوئیچینگ 19 ولت 15
شکل 19-2: رگولاتور سوئیچینگ 19 ولت به همراه قطعات آن 15
شکل 20-2: 20-2 : میکروکنترلر STM32F407VET6 16
شکل 21-2: شماتیک مدار میکروکنترلر STM32F407VET6 17
شکل 22-2: ترتیب و عملکرد پایه های آی سی SDRAM 18
شکل 23-2: نحوه برقراری ارتباط با آی سی SDRAM 19
شکل 24-2: جدول مقادیر ولتاژ و جریان آی سی SDRAM 19
شکل 25-2: بلوک دیاگرام اجزای داخلی آی سی SDRAM 19
شکل 26-2: شماتیک مدار درگاه کارت حافظه 20
شکل 27-2: درگاه کارت حافظه 20
شکل 28-2: شماتیک درگاه نمایشگر 21
شکل 29-2: درگاه نمایشگر 21
شکل 30-2: ماژول گیرنده مادون قرمز 22
شکل 31-2: شماتیک مدار ماژول گیرنده مادون قرمز 22

شکل 32-2: بلوک دیاگرام پروژه 23
شکل 1-3: ایجاد پروژه جدید در نرم افزار STM32CubeIDE 25
شکل 2-3: صفحه انتخاب میکروکنترلر در نرم افزار STM32CubeIDE 25
شکل 3-3: صفحه انتخاب نام پروژه در نرم افزار STM32CubeIDE 26
شکل 4-3: : محیط CubeMX در نرم افزار STM32CubeIDE 26
شکل 5-3: ساختار فایل های برنامه در نرم افزار STM32CubeIDE 27
شکل 6-3: پروگرامر STLINK-V2 42
شکل 1-4: محیط نرم افزار تبدیل فرمت تصویر 44

فصل ۱– کلیات پروژه

1-1- مقدمه :

امروزه نمایشگر های کریستال مایع یا به اختصار LCD [1]، با کاربرد های متخلفی در سراسر دنیا مشغول به کار هستند. صنعت ، آموزش ، تبلیغات، رسانه و ... به لطف وجود این نمایشگر ها میتوانند به راحتی محتوای خود را روی این نمایشگر های به نمایش در بیاورند. اما در پشت پرده این تکنولوژی ، مدارات و سیستم های الکترونیکی پیچیده و مختلفی وجود دارند که با سرعت بالایی در حال کنترل کردن تک تک پیکسل[2] های نمایشگر هستند تا یک تصویر به نمایش در بیاید.

1-2- بیان مسئله پروژه :

مسئله اصلی این در این پروژه، نمایش محتوای تصویری دلخواه روی یک نمایشگر LCD است. چالش اصلی، دسترسی به سرعت بالا در کنترل کردن پیکسل های نمایشگر است. همچنین لازم است تا سیستم ما توانایی برقراری ارتباط با یک کارت حافظه[3] را نیز داشته باشد تا بتواند اطلاعات یک فایل تصویری [4] را از روی آن بخواند و روی نمایشگر نشان دهد. برای جذاب تر شدن کار، یک کنترل مادون قرمز[5] هم به پروژه اضافه میشود.

1-3- اهداف پروژه :

هدف اصلی در طراحی پروژه، پیدا کردن راه حل برای مسائل مطرح شده است. بنابراین باید در ابتدا نحوه کارکرد نمایشگر ها وکارت حافظه را مورد بررسی قرار داده و سپس، سخت افزاری را طراحی کنیم که بتواند اطلاعات یک تصویر را از کارت حافظه بخواند و روی نمایشگر نشان دهد.

---

[1] Liquid crystal display
[2] Pixel
[3] Micro SD Card
[4] Image File
[5] Infra Red Remote

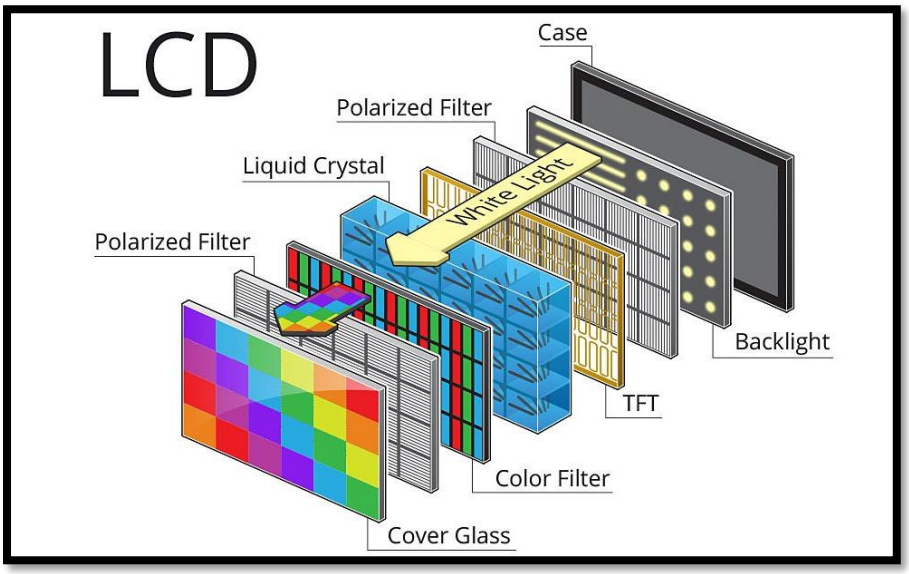
فصل ۲– سخت افزار

2-1- مقدمه :

هسته اصلی سخت افزار این پروژه یک میکروکنترلر ARM[1] ساخته شرکت STMicroelectronics اســـت که در کنار واحد های مختلفی همچون واحد تغذیه، حافظه RAM[2]، درگاه کارت حافظه، درگاه نمایشــگر و ماژول گیرنده مادون قرمز قرار گرفته تا بتواند نیاز های ما را برآورده ســازد. ســخت افزار این پروژه شامل یک PCB[3] و یک نمایشگر TFT-LCD[4] است .

2-2- نمایشگر TFT :

برای نمایش محتوای تصویری از یک نمایشگر 5 اینچی با رزولوشن[5] 480 در 320 پیکسل استفاده شده است. در شکل زیر ساختار داخلی یک نمایشگر TFT-LCD را مشاهده میکنید.



شکل 2-1 : بخش های داخلی یک TFT-LCD

[1] Advanced RISC Machine
[2] Random Acsess Memory
[3] Printed Circuit Board
[4] Thin-film-transistor liquid-crystal display
[5] Resolution

2-2-1- ارتباط با نمایشگر TFT :

اکثر نمایشگر های TFT از کابل های فلت 40 یا 50 پایه برای برقراری ارتباط با واحد کنترل کننده استفاده میکنند. نمایشگر ما دارای کابل فلت 40 پایه است که در ادامه به بررسی عملکرد هریک از آن ها می پردازیم.

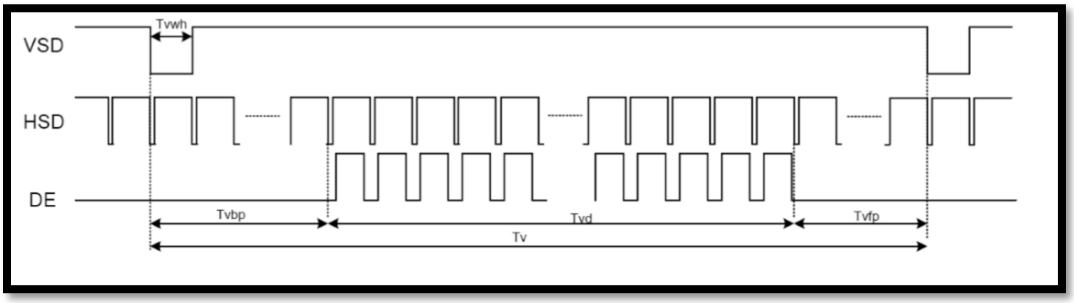
شکل 2-2 : نمایشگر 5 اینچ به همراه کابل فلت 40 پایه

Pin.No	Symbol	Function
1	LEDK	back light power supply negative
2	LEDA	back light power supply positive
3	GND	Ground
4	DVDD	Power supply
5-12	R0-R7	Red Data
13-20	G0-G7	Green Data
21-28	B0-B7	Blue Data
29	GND	Ground
30	CLK	Colock signal
31	DISP	Display on/off
32	HSYNC	Horizontal sync input in RGB mode(short to GND if not used)
33	VSYNC	Vertical sync input in RGB mode(short to GND if not used)
34	DE	Data enable
35	NC	No Connection
36	GND	Ground
37	XR	touch panel X-right
38	YD	touch panel Y-bottom
39	XL	touch panel X-left
40	YU	touch panel Y-upl

شکل 3-2 : تشریح پایه های کابل فلت 40 پایه

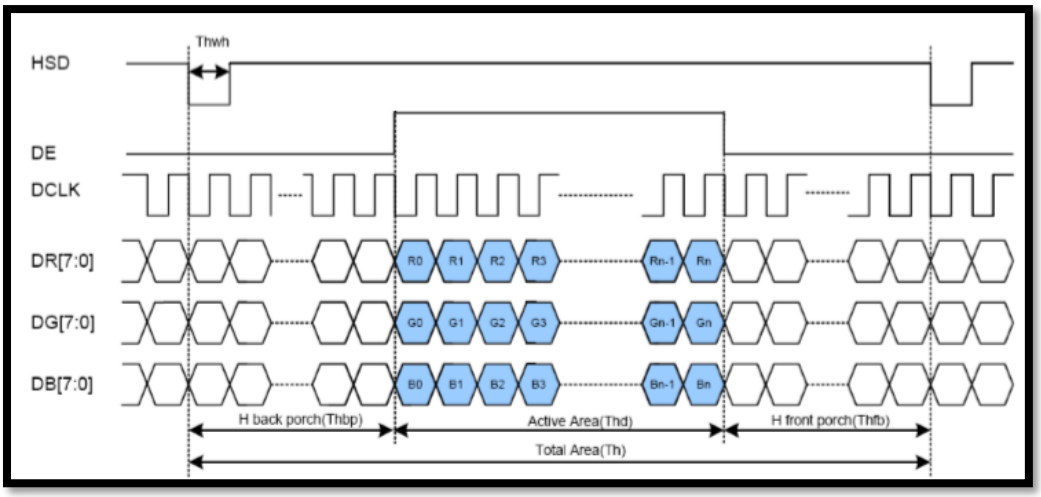
در این نمایشگر ها، اطلاعات هر پیکسل که شامل مقدار رنگ های قرمز(R) سبز(G) و آبی (B) در قالب 3 عدد 8 بیتی هستند به صورت یک جا روی پایه های 5 تا 28 ال سی دی دریافت می شوند. به این پروتکل به اختصار RGB88 گفته میشود. ما در این پروژه برای بالابردن سرعت ارتباط از پروتکل RGB565 استفاده کردیم که برای رنگ قرمز 5 بیت ، رنگ سبز 6 بیت و رنگ آبی 5 بیت پرارزش را عدد دهی میکند و برای بقیه بیت های آن ها عدد صفر در نظر می گیرد. به این ترتیب برای هر پیکسل، به جای 24 بیت ، 16 بیت اطلاعات در نظر گرفته می شود.

در شکل زیر نحوه مقدار دهی پیکسل های نمایشگر را مشاهده میکنید. در هر بار مقدار دهی کل صفحه نمایش (VSD)، باید کل سطر ها را مقدار دهی کرد و در هر بار مقدار دهی یک سطر (HSD) باید کل پیکسل های آن سطر را مقدار دهی نمود.



شکل 4-2 : نحوه مقدار دهی سطر به سطر صفحه نمایش

در هر بار مقدار دهی پیکسل های یک سطر لازم است ابتدا به پایه DE مقدار 1 داده شود. سپس مقدار رنگ های پیکسل اول روی پایه های متناظر قرار داده شود و بعد از آن پایه CLK یک لبه بالارونده داشته باشد. به همین ترتیب برای پیکسل های بعدی مقدار رنگ را روی پایه ها قرار داده و به پایه CLK یک لبه بالا رونده میدهیم. پس از پایان مقدار دهی پیکسل های یک سطر به پایه DE مقدار صفر داده و مجددا HSD را برای سطر بعدی اجرا میکنیم.



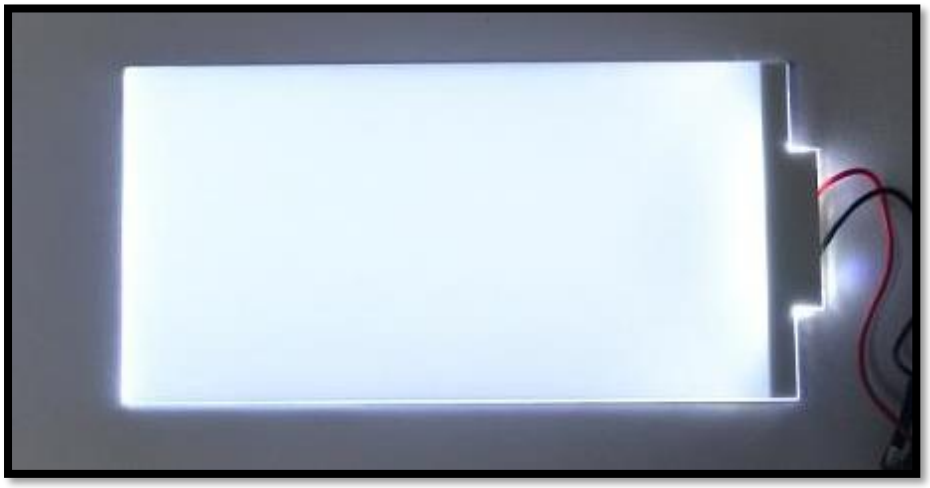
شکل 5-2 : نحوه مقدار دهی پیکسل های یک سطر از صفحه نمایش

2-2-2- نور پس زمینه در نمایشگر TFT :

همانطور که در ساختار نمایشگر های TFT مشاهده شد. یک قسمت تحت عنوان نور پس زمینه در این نمایشگر ها وجود دارد. این قسمت متشکل از چند LED با نور سفید هستند که نور یک دستی را ایجاد میکنند.پایه های 1 و 2 کابل فلت برای تغذیه این قسمت در نظر گرفته شده است. در شکل زیر شماتیک این LED ها و جدول مشخصات فنی آن را میبینید.

Item	Symbol	MIN	TYP	MAX	UNIT	Test Condition
Supply Voltage	Vf	20.3	22.4	24.5	V	If=20mA
Supply Current	If	-	20	30	mA	-
Luminous Intensity for LCM	-	150	200	-	Cd/m^2	If=20mA
Uniformity for LCM	-	80	-	-	%	If=20mA
Life Time	-	20000	-	-	Hr	If=20mA
Backlight Color	White					

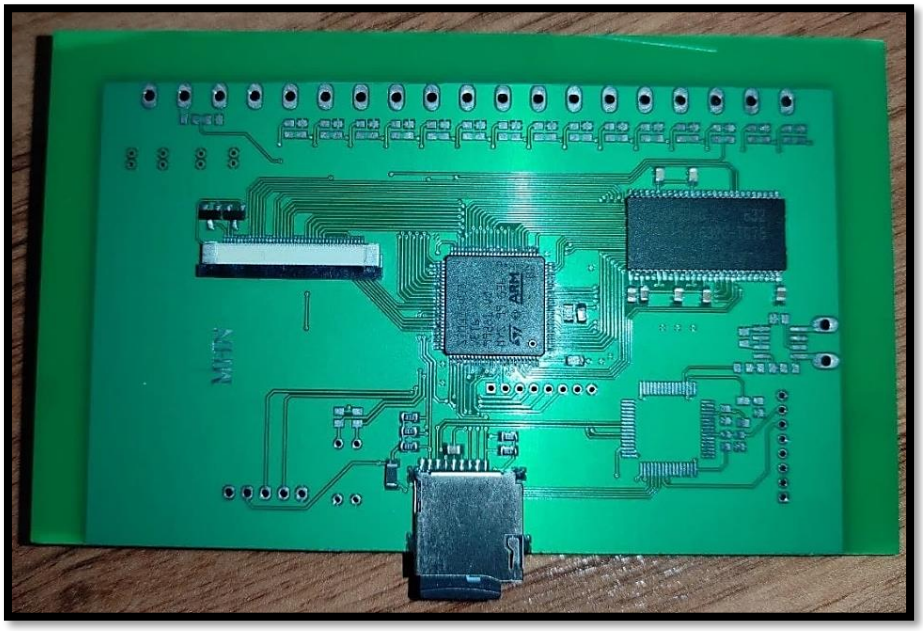
شکل 6-2 : شماتیک و مشخصات فنی بخش نور پس زمینه



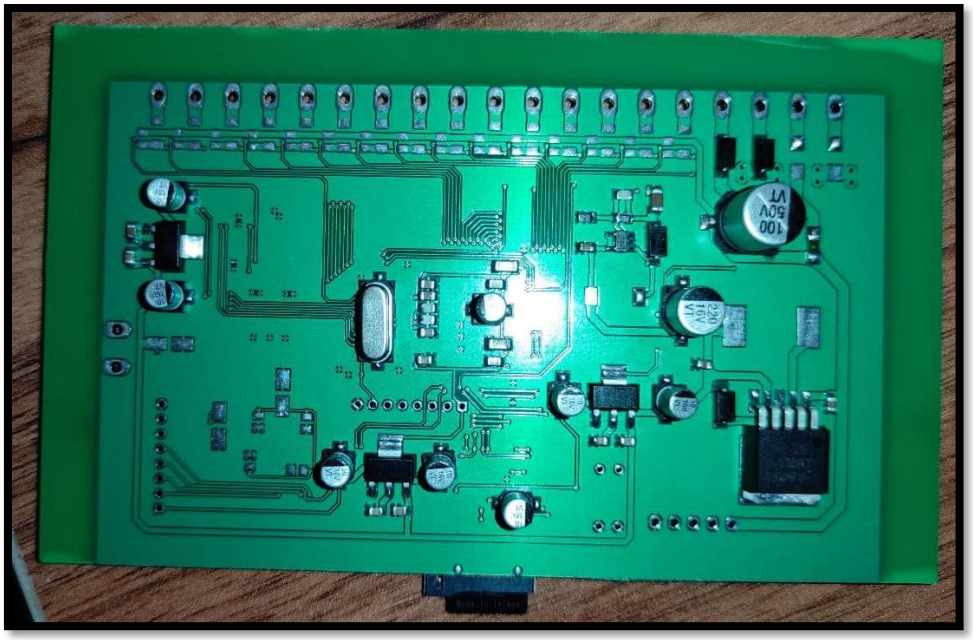
شکل 7-2 : یک نمونه LED نور پس زمینه در حالت روشن

2-3- PCB پروژه :

تمامی بخش های الکترونیکی مورد نیاز پروژه روی یک PCB قرار گرفته اند که در نرم افزار Altium Designer طراحی شده و توسط شرکت های تولید کننده PCB ساخته شده است. این PCB از ورودی DC[1] 8 ولت تا 35 ولت پشتیبانی میکند و قابلیت اتصال نمایشگر های TFT با اندازه 5 اینچی را دارد. در ادامه به معرفی و بررسی واحد های مختلف آن می پردازیم.



شکل 2-8 : PCB پروژه (رو)

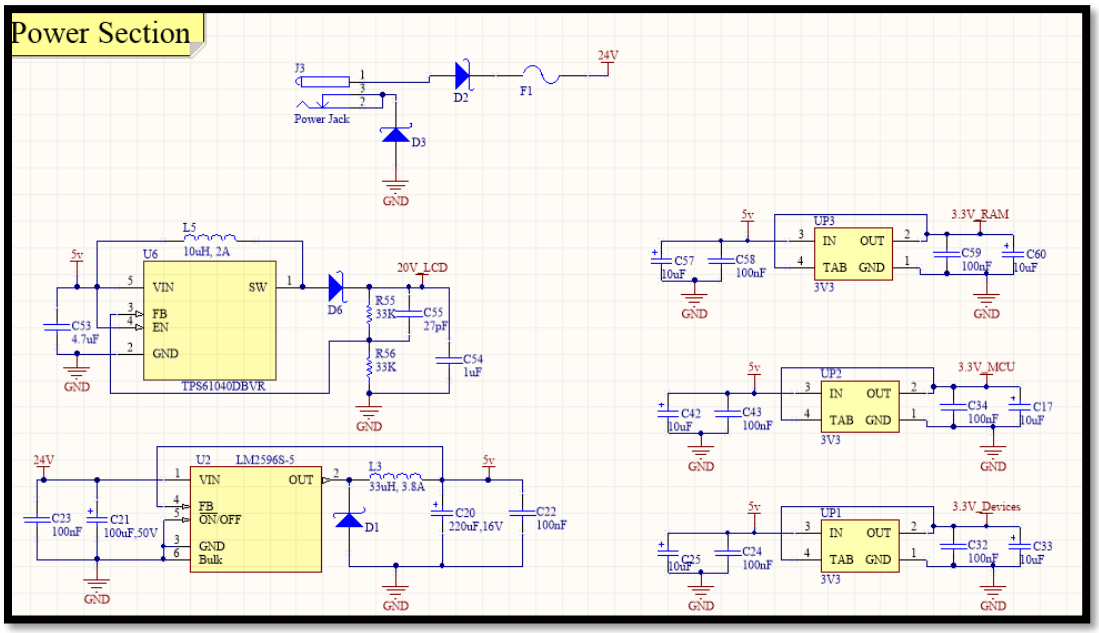


شکل 2-9 : PCB پروژه (پشت)

[1] Direct Current

2-3-1- بخش تغذیه :

یکی از مهم ترین بخش های هر مدار الکترونیکی، بخش تغذیه آن است. دراین پروژه، بخش تغذیه به چند بخش مجزا تقسیم شده که به صورت جداگانه به معرفی و بررسی آن ها خواهیم پرداخت. همچنین شماتیک کلی بخش تغذیه را در شکل زیر می بینید.

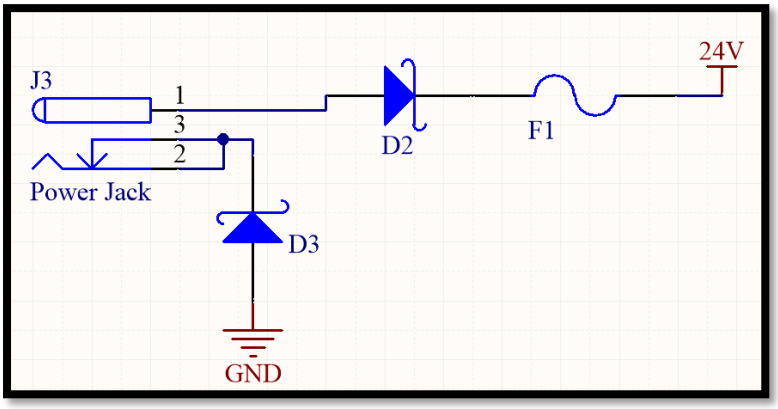


شکل 2-10 : شماتیک کلی بخش تغذیه

2-3-2- مدار حفاظت ورودی تغذیه :

ورودی تغذیه مدار یک جک آداپتوری مادگی است. خطوط تغذیه ورودی ابتدا از دو دیود[1] عبور میکنند تا درصورت اتصال تغذیه با پلاریته معکوس، جریانی از دیود ها عبور نکرده ومدار آسیبی نبیند. همچنین در مسیر ولتاژ مثبت ورودی یک فیوز[2] 1 آمپر قرار دارد تا اگر در صورت بروز مشکل، جریان کشی مدار بالا رفت این فیوز اتصال باز شده و ورودی تغذیه قطع گردد.

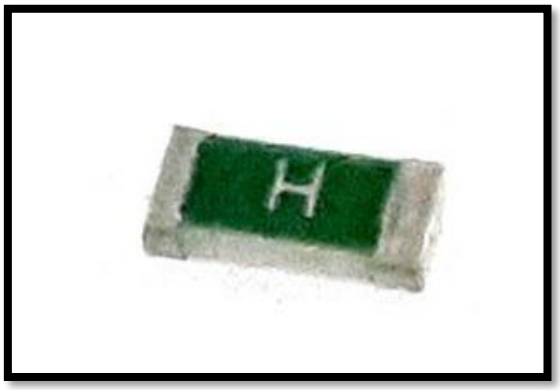
[1] Diode
[2] Fuse



شکل 11-2 : شماتیک بخش ورودی تغذیه

دیود های استفاده شده از نوع شاتکی[1] با پارت نامبر 1N5822 هستند که قابلیت تحمل جریان مستقیم[2] تا 3 آمپر و ولتاژ معکوس[3] تا 40 ولت را دارند. همچنین افت ولتاژ دوسر این دیود در حالت جریان مستقیم 500 میلی ولت است.

شکل 12-2 : دیود SMD - 1N5822

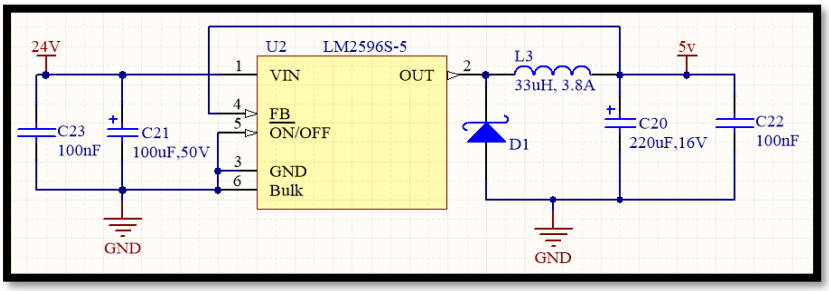


شکل 13-2 : فیوز 1 آمپر SMD

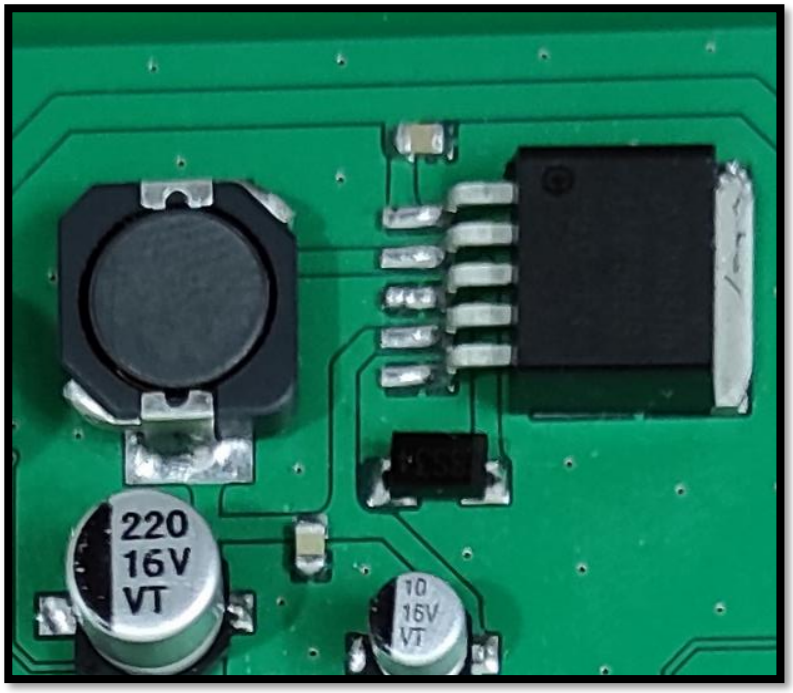
[1] Schottky
[2] Forward Current
[3] Reverse Voltage

2-3-3- رگولاتور سوئیچینگ ۵ ولت :

ولتاژ ورودی پس از عبور از مدارات حفاظت، توسط یک مدار رگولاتور سوئیچینگ[1] به ولتاژ 5 ولت تبدیل میشود که بر پایه آی سی LM2596-5V طراحی شده است. این قطعه یک مبدل سوئیچینگ ولتاژ با خروجی 3 آمپری است که می تواند ورودی 8 تا 40 ولت را به خروجی ثابت 5 ولتی تبدیل کند. این قطعه در دسته رگولاتور های BULK قرار میگیرد و فرکانس سوئیچینگ آن 173 کیلوهرتز است. این رگولاتور در مدار خود به خازن های الکترولیتی[2] و سرامیکی[3] در ورودی و خروجی برای کاهش اثر نویز و ثبات خروجی و همچنین یک سلف[4] آمپر بالا و یک دیود شاتکی 1N5822 نیاز دارد. شماتیک مدار این رگولاتور به صورت زیر است.



شکل 2-14 :شماتیک مدار رگولاتور 5 ولت

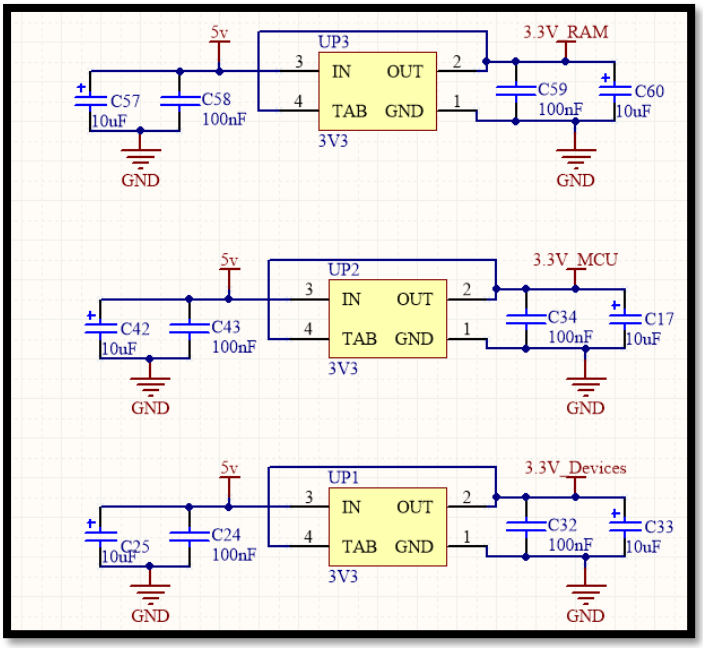


شکل 2-15 :رگولاتور LM2596 درکنار دیگر قطعات مورد نیاز آن

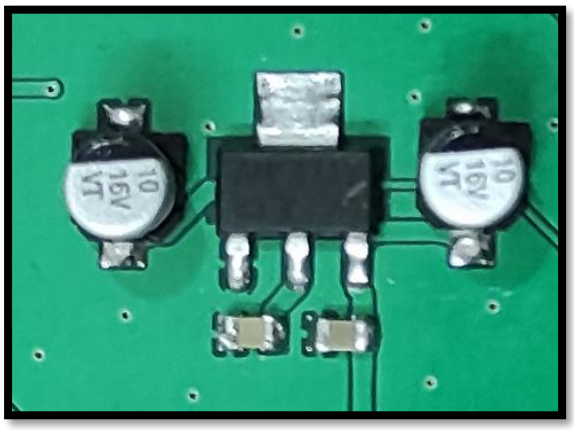
[1] Switching Regulator
[2] Electrolytic
[3] Cramics
[4] Inductor

2-3-4- رگولاتور های خطی ۳.۳ ولت :

پردازنده اصلی، حافظه RAM و کارت حافظه به تغذیه 3.3 ولتی نیاز دارند. برای فراهم کردن تغذیه مورد نیاز آن ها، برای هرکدام، ازمدار رگولاتور خطی[1] AMS1117 با خروجی 3.3 ولت استفاده شده است. این رگولاتور میتواند تا 1 آمپر را در خروجی خود داشته باشد. ورودی این رگولاتور ولتاژ 5 ولت است و در ورودی و خروجی این رگولاتور از خازن های الکترولیت و سرامیکی برای کاهش اثر نویز و ثبات خروجی استفاده شده است.



شکل 2-16 :شماتیک مدار رگولاتور های خطی AMS1117

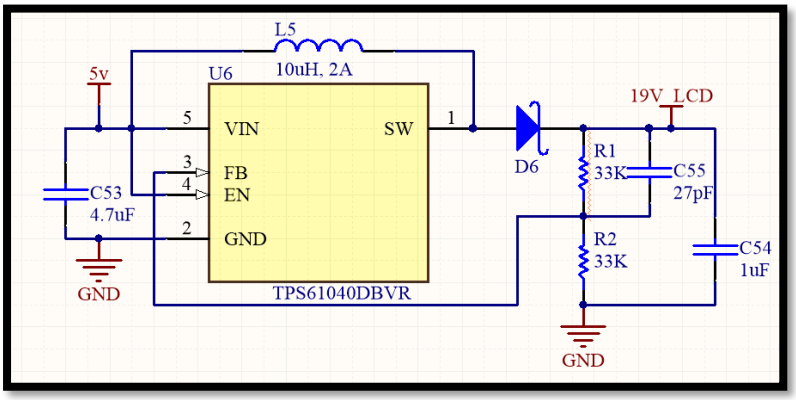


شکل 2-17 : رگولاتور خطی AMS1117 به همراه خازن ها

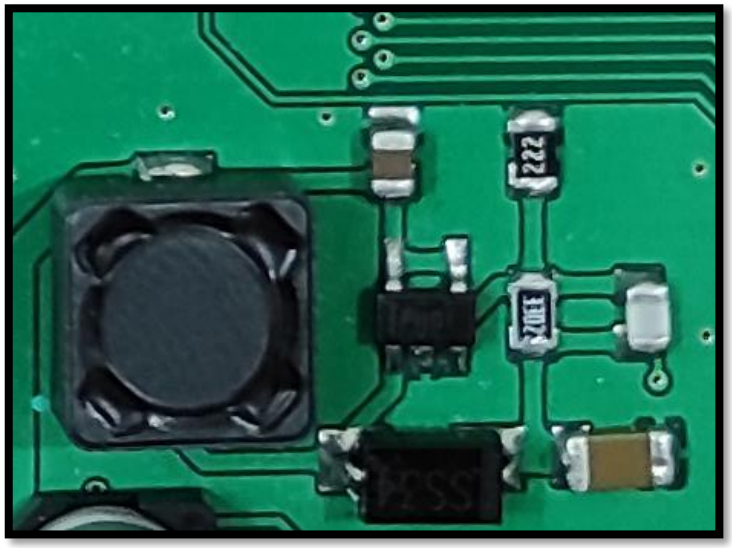
[1] Linear Regulator

5-3-2- رگولاتور سوئیچینگ ۱۹ ولت :

برای تغذیه مدار نور پس زمینه نمایشگر از یک مبدل ولتاژ سوئیچینگ مبتنی بر تراشه TPS61040DBVR استفاده شده است. این قطعه میتواند ولتاژ های 1.8 تا 6 ولت را افزایش دهد و در خروجی خود ولتاژی تا چند برابر ولتاژ ورودی، با حداکثر جریان 400 میلی آمپر داشته باشد. در شکل زیر شماتیک مدار این رگولاتور را مشاهده میکنید.



شکل 18-2 : شماتیک مدار رگولاتور سوئیچینگ 19 ولت

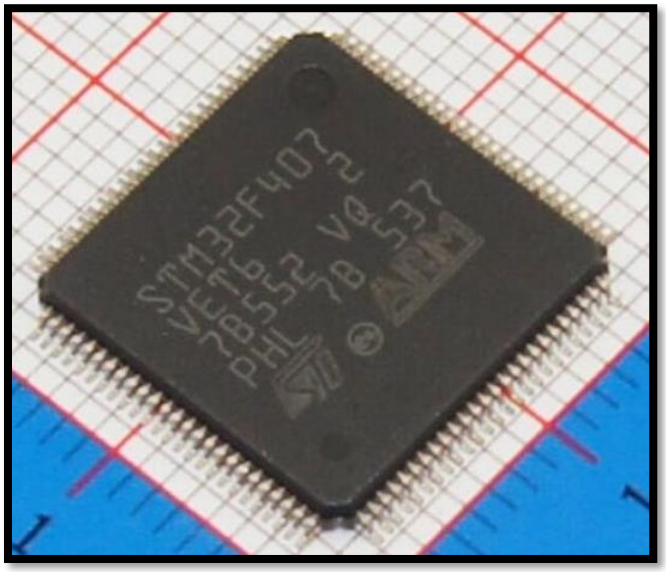


شکل 19-2 : رگولاتور سوئیچینگ 19 ولت به همراه قطعات آن

6-3-2- پردازنده اصلی :

پردازنده اصلی استفاده شده در این پروژه، میکروکنترلر STM32F407VET6 است. هسته اصلی این پردازنده32 بیتی، هسته Cotex-M4 از خانواده هسته هایARM با حداکثر سرعت پردازشی 168 مگاهرتز می باشد. سایر مشخصات و امکانات این تراشه به این صورت است:

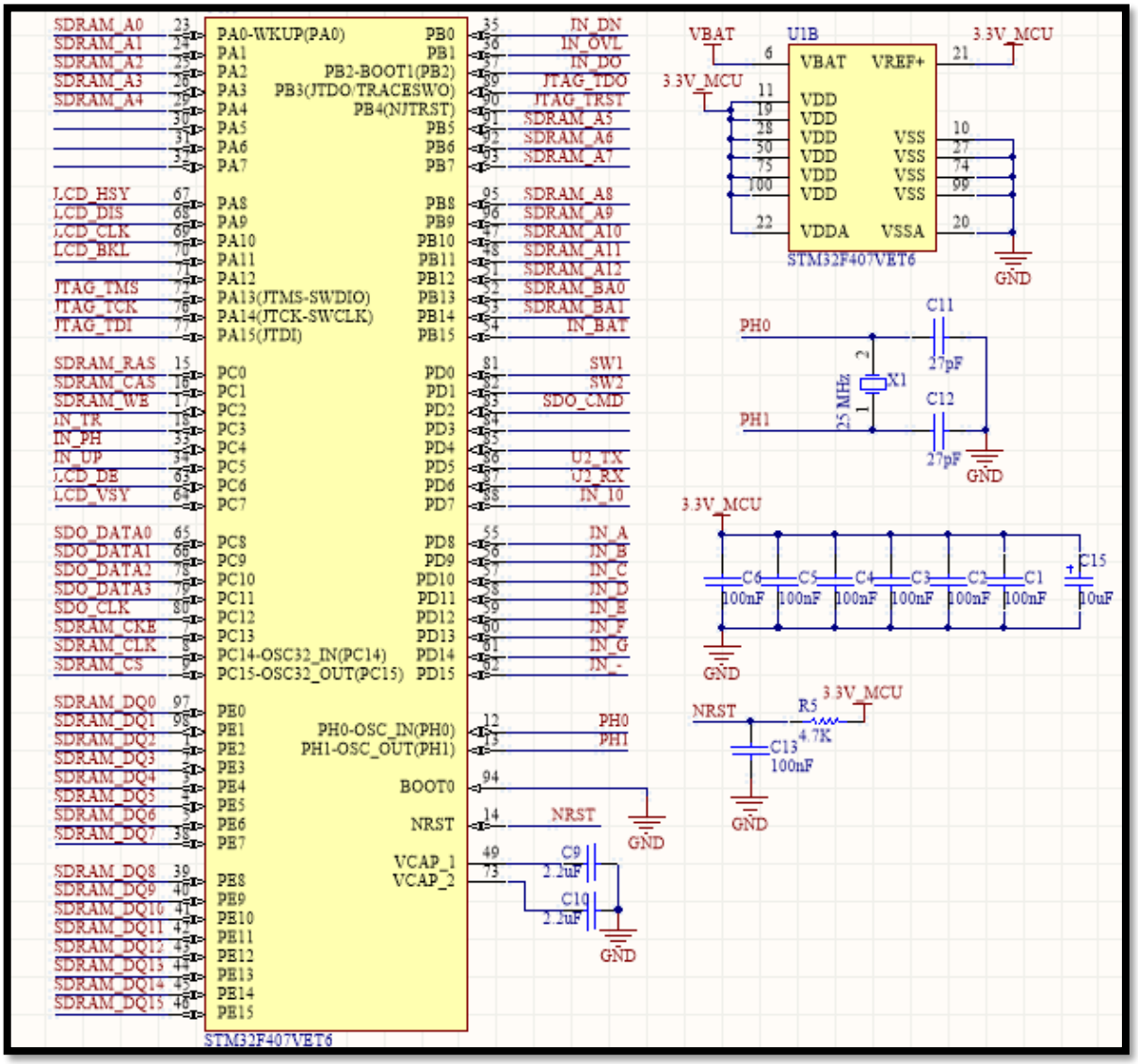
- 192 کیلوبایت حافظه SRAM[1] و 512 کیلوبایت حافظه FLASH
- ولتاژ کاری 1.8 ولت تا 3.6 ولت
- 82 پایه ورودی / خروجی قابل برنامه ریزی
- درگاه های USB[8] / UART [7]/ SPI[6]/ I2S [5]/ SDIO[4]/ I2C[3]/ CAN[2]
- 16 کانال مبدل آنالوگ به دیجیتال با دقت 12 بیت
- 2 کانال مبدل دیجیتال به آنالوگ با دقت 12 بیت
- دارای واحد های پردازشی FPU [۹]و RNG [10]
- کارکرد در دمای 40- تا 85+ درجه سانتی گراد



شکل 20-2 : میکروکنترلر STM32F407VET6

[1] Static Random-Access Memory
[2] Controller Area Network
[3] Inter-Integrated Circuit
[4] Secure Digital Input Output
[5] Inter-IC Sound
[6] Serial Pheripheral Interface
[7] Universal Asynchronous Receiver-Transmitter
[8] Universal Serial Bus
[9] Floating Point Unit
[10] Random Number Generator

در شکل زیر، شماتیک مداری این قطعه و عملکرد تعریف شده برای هر پایه از آن را مشاهده می کنید. این آی سی دارای 6 پایه تغذیه می باشد که همگی آن ها با خازن سرامیکی 100 نانو فارادی موازی شده اند. همچنین یک کریستال 25 مگاهرتز به پایه های ورودی نوسان ساز این آی سی متصل شده و خازن های 27 پیکو فارادی برای تثبیت سیگنال نوسان در کنار کریستال قرار گرفته اند. پایه ریست میکروکنترلر نیز با یک مقاومت 4.7 کیلو به VCC متصل شده ویک خازن 100 نانو فاراد هم برای کاهش اثر نویز روی این پایه قرار گرفته است. روی دو پایه VCAP_1 و VCAP_2 هم خازن های 2.2 میکرو فاراد وجود دارد.



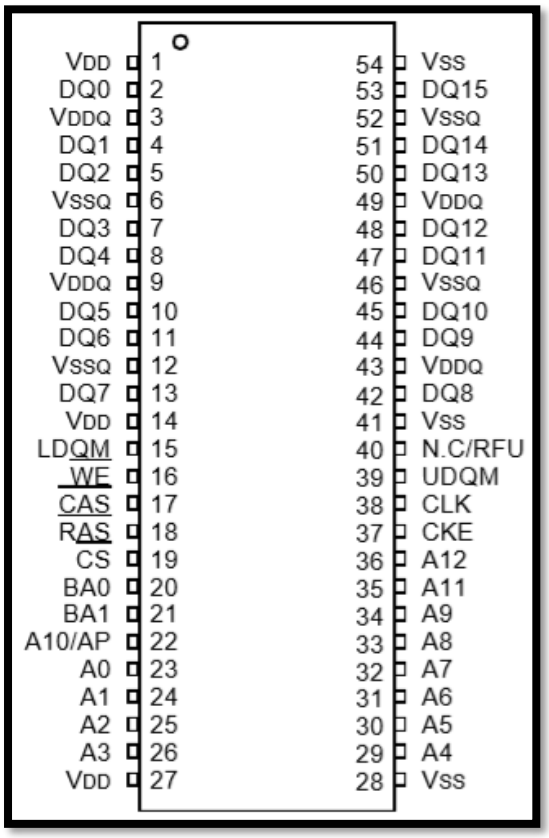
شکل 2-21 : شماتیک مدار میکروکنترلر STM32F407VET6

7-3-2- آی سی RAM :

برای داشتن حافظه ای با طرفیت بالا و درعین حال سریع، از یک آی سی SDRAM[1] ساخت شرکت سامسونگ با پارت نامبر K4S511632B-TC75 استفاده شده است. حافظه این قطعه 256 مگابیت معادل 32 مگابایت است که در 4 بانک[2] تقسیم شده است. هر بانک شامل 4,194,304 خانه حافظه 16 بیتی است که به صورت ماتریسی در 4192 سطر[3] و 512 ستون[4] قرار دارند.

این آی سی از درگاه DDR[5] که پهنای داده 16 بیتی دارد برای تبادل داده و آدرس دهی استفاده میکند. 16 پایه از آی سی(DQ0-DQ15) برای تبادل اطلاعات در جهت ورودی و خروجی استفاده شده و 13 پایه از آن(A0-A12)، مشترکاً برای آدرس دهی سطرها و ستون ها استفاده می شود. در شکل زیر عملکرد هر پایه از این قطعه را مشاهده می کنید.

Pin	Name
CLK	*System cock*
$\overline{CS}$	*Chip select*
CKE	*Clock enable*
A_0 ~ A_{12}	*Address*
BA_0 ~ BA_1	*Bank select address*
$\overline{RAS}$	*Row address strobe*
$\overline{CAS}$	*Column address strobe*
$\overline{WE}$	*Write enable*
L(U)DQM	*Data input/output mask*
$DQ_{0 \sim 15}$	*Data input/output*
V_{DD}/V_{SS}	*Power supply/ground*
V_{DDQ}/V_{SSQ}	*Data output power/ground*
N.C/RFU	*No connection /reserved for future use*



شکل 22-2 : ترتیب و عملکرد پایه های آی سی SDRAM

[1] Synchronous Dynamic Random-Access Memory
[2] Bank
[3] Row
[4] Column
[5] Double Data Rate

برای برقرای ارتباط با این قطعه و همچنین خواندن و نوشتن اطلاعات می بایست طبق جدول زیر به پایه های این آی سی سیگنال های کنترلی اعمال کرد.

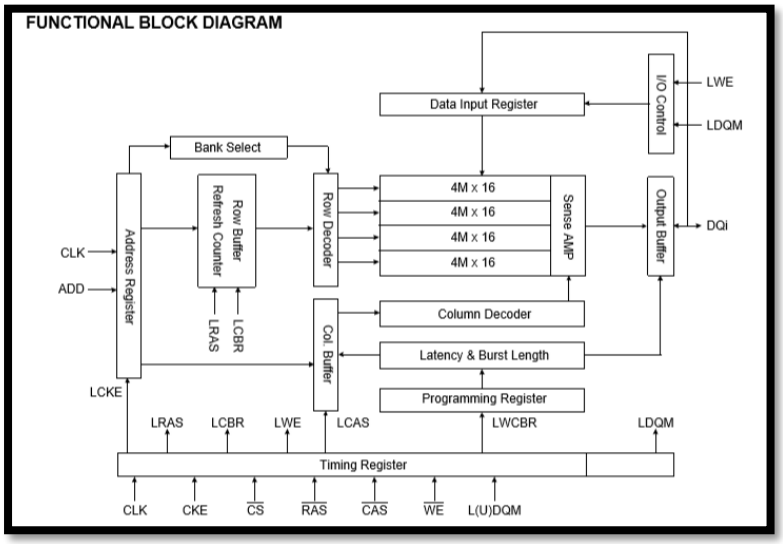
Command			CKEn-1	CKEn	$\overline{CS}$	$\overline{RAS}$	$\overline{CAS}$	$\overline{WE}$	DQM	$BA_{0,1}$	A_{10}/AP	A_{11},A_{12}, A_9 ~ A_0	Note
Register	Mode register set		H	X	L	L	L	L	X	OP code			1,2
Refresh	Auto refresh		H	H	L	L	L	H	X	X			3
	Self refresh	Entry		L									3
		Exit	L	H	L	H	H	H	X	X			3
					H	X	X	X					3
Bank active & row addr.			H	X	L	L	H	H	X	V	Row address		
Read & column address	Auto precharge disable		H	X	L	H	L	H	X	V	L	Column address (A_0 ~ A_9)	4
	Auto precharge enable										H		4,5
Write & column address	Auto precharge disable		H	X	L	H	L	L	X	V	L	Column address (A_0 ~ A_9)	4
	Auto precharge enable										H		4,5
Burst stop			H	X	L	H	H	L	X	X			6
Precharge	Bank selection		H	X	L	L	H	L	X	V	L	X	
	All banks									X	H		
Clock suspend or active power down		Entry	H	L	H	X	X	X	X	X			
					L	V	V	V					
		Exit	L	H	X	X	X	X	X				
Precharge power down mode		Entry	H	L	H	X	X	X	X	X			
					L	H	H	H					
		Exit	L	H	H	X	X	X	X				
					L	V	V	V					
DQM			H	X					V	X			7
No operation command			H	X	H	X	X	X	X	X			
					L	H	H	H					

(V=Valid, X=Don't care, H=Logic high, L=Logic low)

شکل 23-2 : نحوه برقراری ارتباط با آی سی SDRAM

Parameter	Symbol	Min	Typ	Max	Unit
Supply voltage	V_{DD}, V_{DDQ}	3.0	3.3	3.6	V
Input logic high voltage	V_{IH}	2.0	3.0	V_{DD}+0.3	V
Input logic low voltage	V_{IL}	-0.3	0	0.8	V
Output logic high voltage	V_{OH}	2.4	-	-	V
Output logic low voltage	V_{OL}	-	-	0.4	V
Input leakage current	I_{LI}	-10	-	10	uA

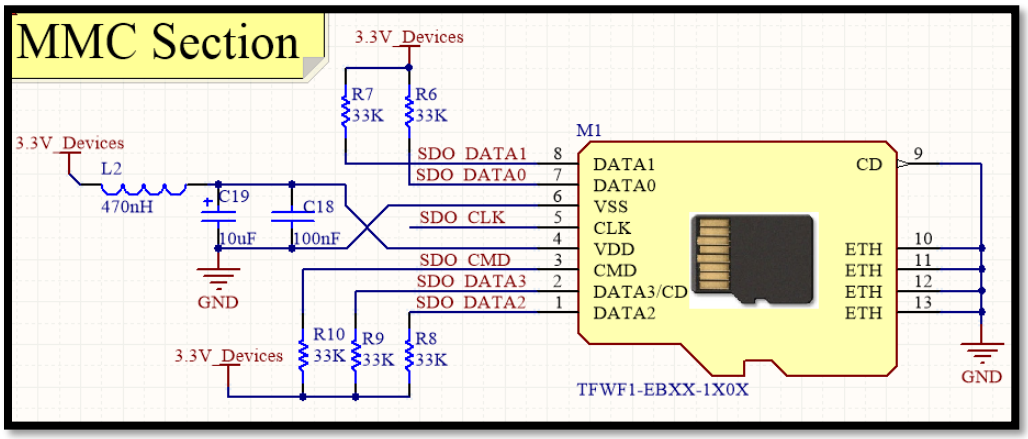
شکل 24-2 : جدول مقادیر ولتاژ و جریان آی سی SDRAM



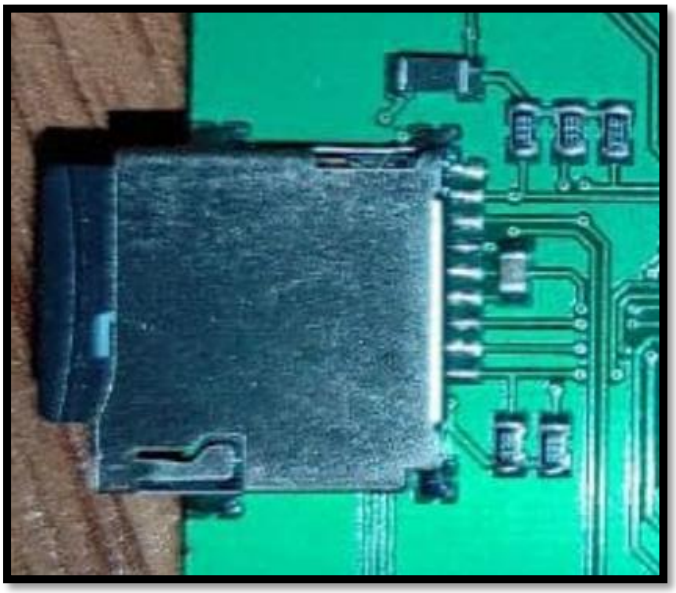
شکل 25-2 : بلوک دیاگرام اجزای داخلی آی سی SDRAM

8-3-2- درگاه کارت حافظه:

اتصال کارت حافظه به میکروکنترلر از طریق یک سوکت میکرو اس دی که روی برد اصلی قرار دارد برقرار می شود. ارتباط با کارت حافظه به دو روش تک بیتی و 4 بیتی انجام می شود که در این پروژه از روش 4 بیتی استفاده شده است. در ارتباط 4 بیتی، پایه های DATA0 تا DATA3 برای تبادل اطلاعات، پایه CLK به برای خط کلاک ارتباط و پایه CMD برای تعیین دستور ارتباطی استفاده می شود. در شکل زیر شماتیک درگاه کارت حافظه را مشاهده می کنید.



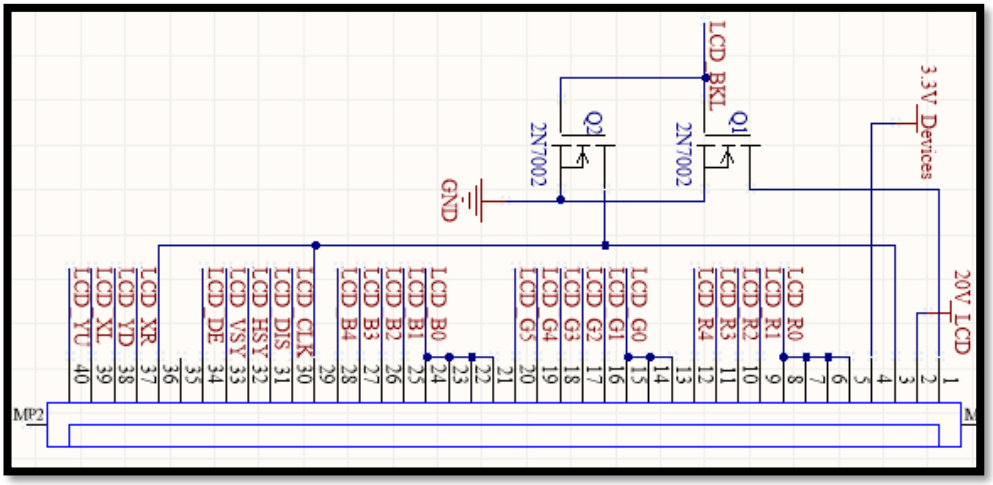
شکل 2-26 : شماتیک مدار درگاه کارت حافظه



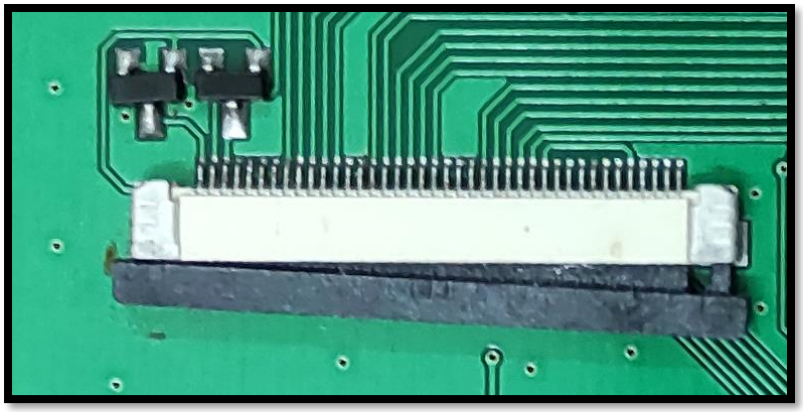
شکل 2-27 : درگاه کارت حافظه

9-3-2- درگاه نمایشگر :

برای اتصال نمایشگر با کابل فلت **۴۰**پین، یک سوکت FPC[1] روی برد اصلی تعبیه شده که اتصال نمایشگر به برد را فراهم می کند. همچنین برای کنترل تغذیه نمایشگر و تغذیه نور پس زمینه، دو عدد ترانزیستور 2N7002 در مدار آن قرار گرفته است که توسط یکی از پایه های میکروکنترلر تحریک می شوند تا بتوانیم نمایشگر را خاموش و روشن کنیم. شماتیک مدار این درگاه را در تصویر زیر مشاهده می کنید.



شکل 28-2 : شماتیک درگاه نمایشگر

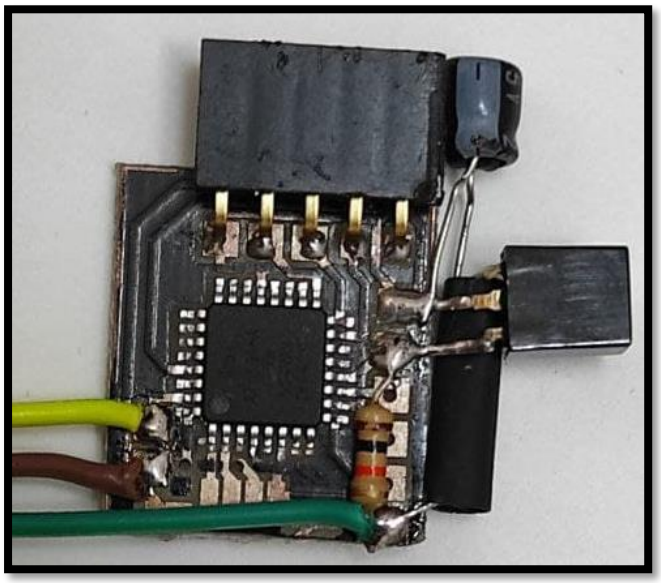


شکل 29-2 : درگاه نمایشگر

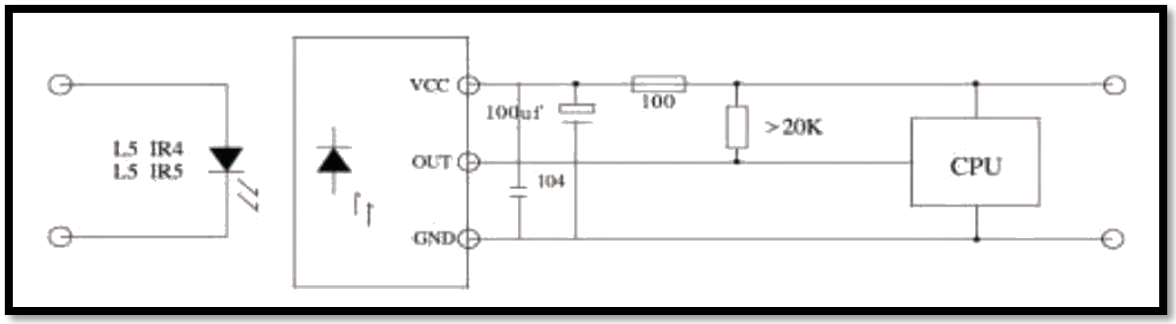
[1] Flat Printed Circuit

2-3-10- ماژول گیرنده مادون قرمز :

برای دیکود کردن اطلاعات دریافتی از ریموت های مادون قرمز از یک ماژول گیرنده مادون قرمز استفاده می شود. این ماژول متشکل از یک مدار سنسور گیرنده امواج مادون قرمز و یک میکروکنترلر AVR است که پس از دریافت اطلاعات در قالب پروتکل RC6 ، کاراکتر های دریافت شده را از طریق پورت سریال به میکروکنترلر اصلی ارسال می کند. این ماژول به صورت جداگانه برای پروژه های قبلی طراحی شده بود که برحسب نیاز، در این پروژه هم استفاده شده است.



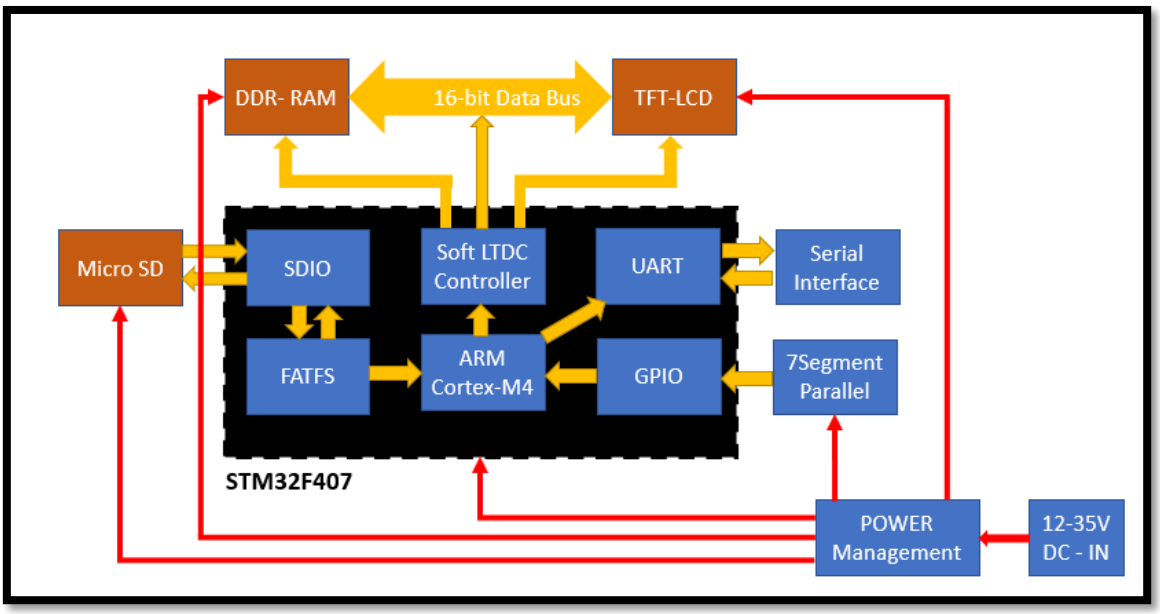
شکل 30-2 : ماژول گیرنده مادون قرمز



شکل 31-2 : شماتیک مدار ماژول گیرنده مادون قرمز

2-4- نتیجه گیری:

پروژه ساخته شده شامل یک برد اصلی و یک نمایشگر TFT است. هسته مرکزی برد اصلی را یک میکروکنترلر ARM تشکیل می دهد. تمامی درگاه ها و قطعات سیستم تغذیه روی این برد قرار دارند. قطعات استفاده شده روی این برد، پس از نیاز سنجی ابتدایی و بعد از تحقیق و بررسی درمورد مدارها و قطعات مختلف الکترونیکی انتخاب شده اندوطراحی شماتیک وPCBتوسط نرم افزار Altium Designer انجام شده است.



شکل 2-32 : بلوک دیاگرام پروژه

فصل ۳– نرم افزار

3-1- مقدمه :

برای برنامه نویسی میکرو کنترلر های ARM می توان از کامپایلر[1] هایی نظیر Keil یا IAR و همچنین محیط توسعه نرم افزاری شرکت STM یعنی STM32CubeIDE که برای میکروکنترلرهای سری STM32 ساخت این شرکت ارائه شده استفاده نمود که انتخاب ما برای این پروژه است. دلیل انتخاب این کامپایلر داشتن قابلیت های متنوعی نظیر محیط CubeMX که برای ساخت خودکار کد ها و ساختار فایل های برنامه نویسی به صورت پیش فرض استفاده می شود و همچنین قابلیت Debug کردن و آنالیز کردن برنامه است. پس از تکمیل برنامه نویسی و کامپایل[2] کردن آن، با یک دستگاه پروگرامر، فایل خروجی کامپایل شده را به میکروکنترلر منتقل می کنیم.

3-2- معرفی نرم افزار STM32CubeIDE :

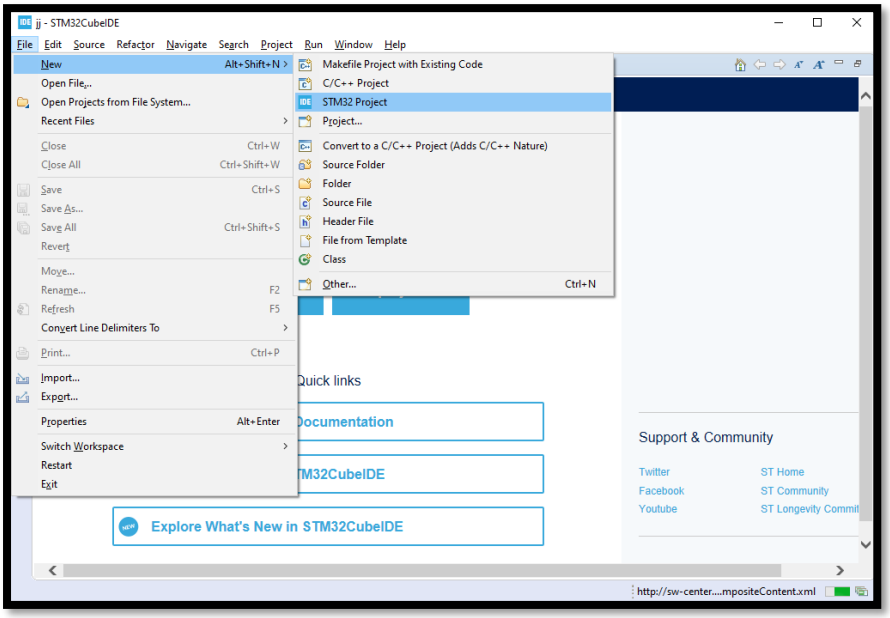
این نرم افزار در واقع محیطی ویرایش شده از ادیتور[3] قدرتمند Eclipse است که به صورت تخصصی برای توسعه نرم افزاری میکروکنترلر های STM32 ارائه شده است. شرکت STMicroelectronic این نرم افزار را به صورت رایگان در سایت خود قرار داده و به صورت دوره ای آن را به روز رسانی می کند. همچنین تمامی پکیج ها و کتابخانه های نرم افزاری مورد نیاز از جمله رابط برنامه نویسی HAL[4] ، کتابخانه سیستم فایل FATFS [5]، توابع CMSIS و همچنین سیستم عامل بلادرنگ FreeRTOS[6] هم در این نرم افزار پشتیبانی می شوند. در این نرم افزار میتوان با زبان های C و C++ برای میکروکنترلر خود برنامه نویسی کرد و با داشتن یک دستگاه دیباگر[7] ، برنامه نوشته شده را در حین اجرا بررسی کرد و مقدار متغیر ها و وضعیت حافظه ها را مشاهده کرد. این قابلیت باعث کاهش چشمگیر زمان پیداکردن خطاهای احتمالی در برنامه نویسی می شود. اگرچه تاکنون گزارش هایی مبنی بر وجود خطا در برخی نسخه های این نرم افزار گزارش شده و من هم تاکنون به چندین مورد خطا در این نرم افزار برخورد کرده ام اما شخصا این نرم افزار را ترجیح می دهم.

3-3- ایجاد پروژه جدید در STM32CubeIDE :

پس از دریافت فایل نصبی نرم افزار و نصب آن ، نوبت به اجرا کردن نرم افزار و ایجاد پروژه جدید می رسد. مطابق شکل از منوی File > New > STM32 Project را انتخاب می کنیم.

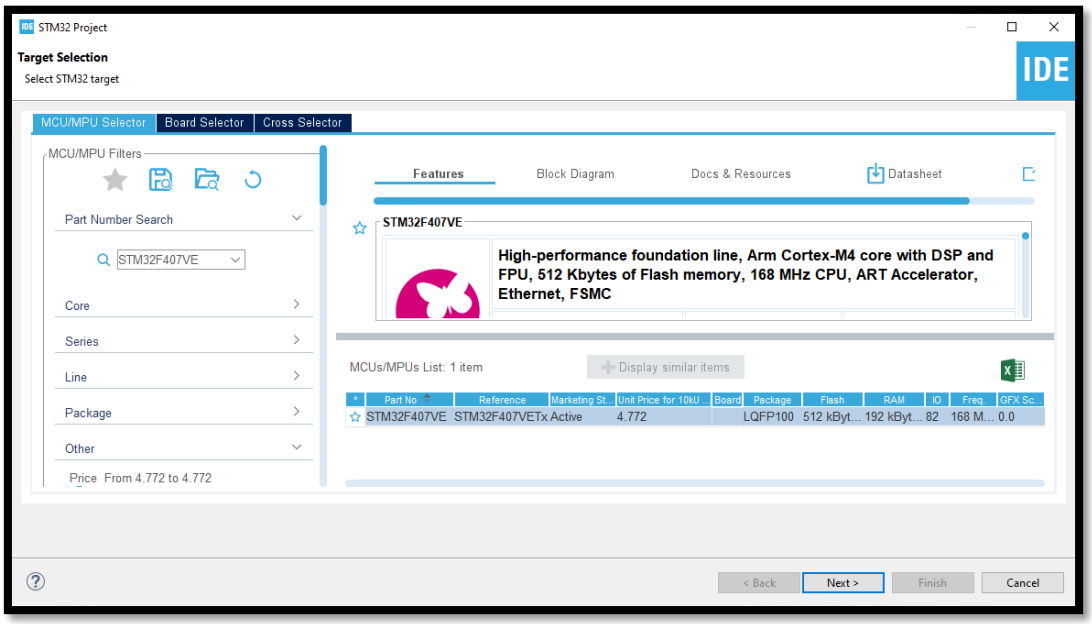
---

[1] Compiler
[2] Compile
[3] Editor
[4] Hardware Abstraction Layer
[5] File Allocation Table File System
[6] Free Real Time Operating System
[7] Debugger



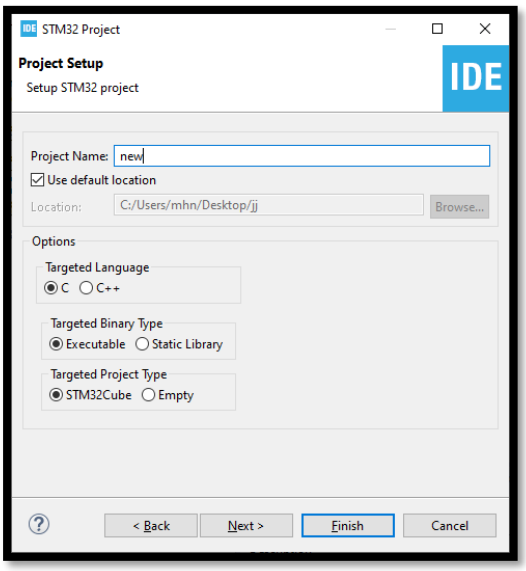
شکل 3-1 : ایجاد پروژه جدید در نرم افزار STM32CubeIDE

پس از انتخاب این گزینه، صفحه ای برای انتخاب میکروکنترلر مورد نظر باز می شود. در کادر Part Number Search پارت نامبر میکروی خود را تایپ میکنیم سپس آن را انتخاب کرده و روی Next کلیک می کنیم. همچنین دراین صفحه، خلاصه ای از امکانات میکروکنترلر به نمایش در می آید.



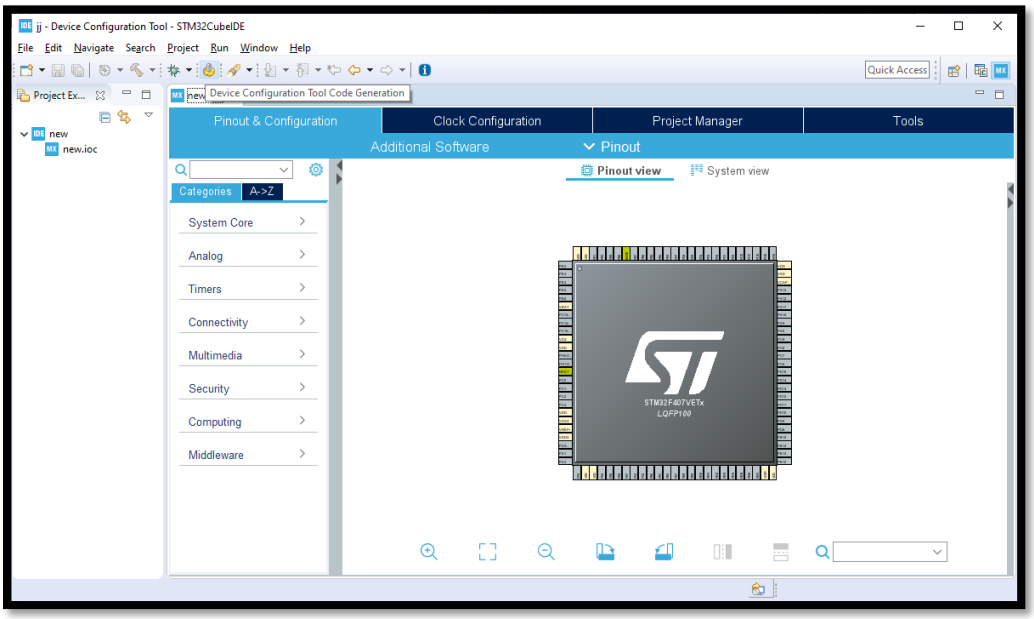
شکل 3-2 : صفحه انتخاب میکروکنترلر در نرم افزار STM32CubeIDE

در صفحه جدیدی که باز می شود، نام پروژه، محل ذخیره سازی و همچنین زبان مورد نظر برای برنامه نویسی را انتخاب کرده وروی Finish کلیک میکنیم.



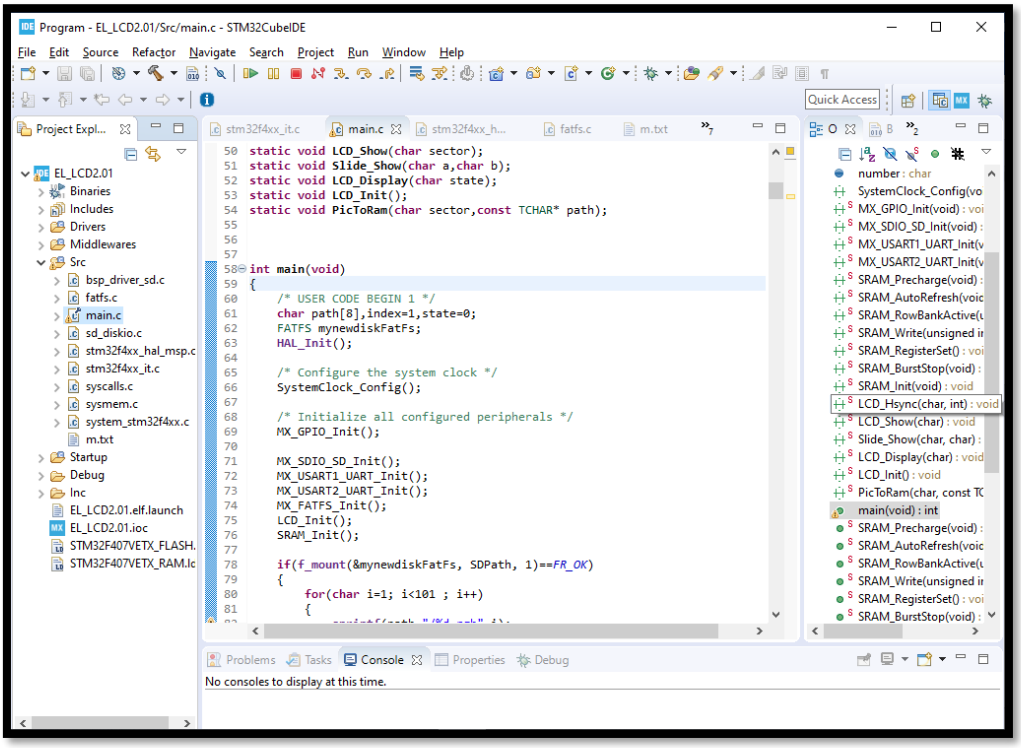
شکل 3-3 : صفحه انتخاب نام پروژه در نرم افزار STM32CubeIDE

محیط CubeMX برای ما به نمایش در می آید. در قسمت Pinout & Configuration عملکرد هر پایه از میکروکنترلر وتنظیمات مربوط به واحد های داخلی، وقفه ها، درگاه ها و پکیج های نرم افزاری مختلف قابل برنامه ریزی هستند. در قسمت Clock Configuration هم میتوان منابع کلاک و فرکانس کلاک واحد های مختلف را برنامه ریزی کرد. پس از اتمام کار خود در این بخش روی گزینه Device Configuration Tool Code Generation کلیک میکنیم تا عملیات ایجاد فایل های برنامه آغاز شود.



شکل 3-4 : محیط CubeMX در نرم افزار STM32CubeIDE

پس از اینکه فایل های برنامه ایجاد شدند از منوی سمت راست و پوشه Src فایل main.c را باز می کنیم.این فایل حاوی کد های اصلی برنامه و همچنین تابع main می باشد. کد نویسی وقفه ها نیز در فایل stm32f4xx_it.c قرار می گیرند.



شکل 5-3 : ساختار فایل های برنامه در نرم افزار STM32CubeIDE

3-4- تشریح کد های برنامه :

برنامه این پروژه در قالب چندین تابع مختلف نوشته شده که به معرفی هر کدام از آن ها می پردازیم. در برنامه نویسی خانواده C ، اجرای برنامه از تابع main آغاز می شود. این تابع حاوی کد های اصلی برنامه ماست که برخی از آن کد ها، توابعی هستند که در آنها زیر کد های آن تابع نوشته شده است.

3-4-1- تابع اصلی برنامه (main) :

کد های برنامه در تابع main شامل دو قسمت هستند. قسمت اول، کد هایی برای پیکره بندی بخش های مختلف میکروکنترلر است و فقط یک بار درشروع اجرای برنامه اجرا می شوند. قسمت دوم، حلقه اصلی (While) است که در آن عملکرد دائمی میکروکنترلر برنامه نویسی می شود و به صورت مداوم تا بی نهایت اجرا می شود.

- بخش اول تابع main :

در این بخش توابع مربوط به پیکره بندی بخش های مختلف و همچنین تعریف متغیر های مورد نیاز انجام می شود.

```c
int main(void)
{
	/* USER CODE BEGIN 1 */
	char path[8],index=1,state=0;
	FATFS mynewdiskFatFs;
	HAL_Init();

	/* Configure the system clock */
	SystemClock_Config();

	/* Initialize all configured peripherals */
	MX_GPIO_Init();

	MX_SDIO_SD_Init();
	MX_USART1_UART_Init();
	MX_USART2_UART_Init();
	MX_FATFS_Init();
	LCD_Init();
	SRAM_Init();
```

- در اینجا اول بررسی می شود که ارتباط سیستم فایل با کارت حافظه برقرار است یا نه سپس در یک حلقه for ، آدرس تصاویر با شماره 1 تا 100 را در متغیر path ایجاد کرده و با تابع PicToRam آن تصویر را در سکتور های شماره 1 تا 100 حافظه RAM قرار می دهیم.

```c
	if(f_mount(&mynewdiskFatFs, SDPath, 1)==FR_OK)
	{
		for(char i=1; i<101 ; i++)
		{
			sprintf(path,"/%d.rgb",i);
			PicToRam(i, path);
		}
	}

	LCD_Display(1);
	__HAL_UART_ENABLE_IT(&huart2, UART_IT_RXNE);
```

- بخش دوم تابع main (حلقه اصلی)

در این حلقه همواره مقدار متغیر State که برای بیان حالت سیستم در نظر گرفته شده بررسی می شود. اگر مقدار آن برابر 0 باشد بیانگر حالت معمولی، 1 بیانگر حالت اسلاید و 2 بیانگر حالت خواب است. در ابتدا مقدار متغیر state برابر 0 است. متغیر دیگری به نام index شماره تصویری که نمایش داده می شود را در خود ذخیره میکند و متغیر receive محتوای دریافتی از درگاه UART که به ماژول گیرنده مادون قرمز متصل است را در خود جای می دهد. متغیر number هم تعداد کل تصاویر را نشان می دهد.

- در حالت معمولی، با دستور LCD_Show(index) تصویر به نمایش در می آید. سپس مقدار متغیر receive بررسی می شود تا اگر مقداری غیر از صفر داشت (روی ریموت کنترل کلیدی فشار داده شد) به دستور داده شده رسیدگی شود. اگر مقداردریافتی از گیرنده برابر U باشد بیانگر تغییر تصویر رو به بالا است .در این صورت اگر شماره تصویر فعلی از تعداد کل تصاویر کمتر بود، تصویر بعدی به نمایش در می آید وبه متغیر index یک واحد اضافه می شود. اگر مقداردریافتی از

گیرنده برابر D باشد بیانگر تغییر تصویر رو به پایین است .در این صورت اگر شماره تصویر فعلی از یک بزرگتر بود ، تصویر قبلی به نمایش در می آید و از متغیر index یک واحد کاسته می شود. اگر مقداردریافتی از گیرنده برابر G باشد مقدار متغیر state برابر 1 قرار داده می شود تا سیستم به حالت اسلاید برود. اگر مقداردریافتی از گیرنده برابر P باشد مقدار متغیر state برابر 2 قرار داده شده و نمایشگر خاموش می شود تا سیستم به حالت خواب برود. در پایان هم متغیر receive برابر صفر قرار داده می شود تا سیستم منتظر دریافت بعدی بماند.

```
while (1)
{

        if(state==0)                    //normal state
        {
                LCD_Show(index);
                if(receive!=0)
                {
                        if(receive=='U' && index<number)
                        {
                                Slide_Show(index, index+1);
                                index++;

                        }
                        else if(receive=='D' && index>1)
                        {
                                Slide_Show(index, index-1);
                                index--;
                        }
                        else if(receive=='G') state=1;
                        else if(receive=='P')
                        {
                                LCD_Display(0);
                                state=2;
                        }
                        receive=0;
                }
        }
```

- در حالت اسلایدر اگر مقدار index از تعداد کل تصاویر کمتر باشد عکس بعدی با حالت اسلایدر به نمایش در می آید و یک واحد به index اضافه می شود. در غیر این صورت (رسیدن به آخر لیست تصاویر) تصویر اول به نمایش در می آید و مقدار index برابر 1 می شود. سپس مقدار متغیر receive بررسی می شود تا اگر مقداری غیر از صفر داشت (روی ریموت کنترل کلیدی فشار داده شد) به دستور داده شده رسیدگی شود. اگر مقداردریافتی از گیرنده برابرR باشد مقدار متغیر state برابر 0 قرار داده می شود تا سیستم به حالت معمولی برود. اگر مقداردریافتی از گیرنده برابر P باشد مقدار متغیر state برابر 2 قرار داده شده و نمایشگر خاموش می شود تا سیستم به حالت خواب برود. در پایان هم متغیر receive برابر صفر قرار داده می شود تا سیستم منتظر دریافت بعدی بماند. سپس برای 300 مرتبه تصویر فعلی به نمایش در می آید. با توجه به این که نرخ نمایش تصاویر در این سیستم 75 تصویر در ثانیه است. عکس فعلی به مدت 4 ثانیه ثابت میماند و پی از آن عکس بعدی به نمایش در می آید.

```
        else if(state==1)                       //slider state
```

```
{
        if(index<number)
        {
                Slide_Show(index, index+1);
                index++;
        }
        else
        {
                Slide_Show(index,1);
                index=1;
        }

        if(receive!=0)
        {
                if(receive=='R') state=0;
                else if(receive=='P')
                {
                        LCD_Display(0);
                        state=2;
                }
                receive=0;
        }
        for(int n=0;n<300;n++)
        {
                LCD_Show(index);
                if(receive!=0)
                {
                        if(receive=='R') state=0;
                        else if(receive=='P')
                        {
                                LCD_Display(0);
                                state=2;
                        }
                        receive=0;
                }
        }

}
```

- در حالت خواب دستورات مربوط به تازه سازی حافظه RAM اجرا می شوند تا اطلاعات از دست نرود. سپس اگر مقدار دریافتی از ریموت برابر P بود نمایشگر روشن می شود و متغیر state برابر صفر شده تا سیستم به حالت معمولی برگردد.

```
-       else if(state==2)                       //Sleep state
        {

                SRAM_Precharge();
                SRAM_AutoRefresh();
                if(receive=='P')
                {
                        LCD_Display(1);
                        state=0;
                        receive=0;
                }

        }
    }
}
```

3-4-2- تابع SRAM_Precharge:

این تابع، عملیات Precharge را روی حافظه RAM اعمال می کند:

```
static void SRAM_Precharge(void)
{
	GPIOC->BSRR=0xC0052002;
	GPIOC->BSRR=0xC0052002;
	GPIOC->BSRR=0xC0052002;
	GPIOB->BSRR=0x400;
	GPIOB->BSRR=0x400;
	GPIOB->BSRR=0x400;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
}
```

3-4-3- تابع SRAM_AutoRefresh:

این تابع، دستور تازه سازی اطلاعات ذخیره شده حافظه RAM را صادر می کند:

```
static void SRAM_AutoRefresh(void)
{
	GPIOC->BSRR=0xC0032004;
	GPIOC->BSRR=0xC0032004;
	GPIOC->BSRR=0xC0032004;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
```

```
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
}
```

3-4-4- تابع SRAM_RowBankActive:

این تابع یک سطر و بانک از حافظه RAM را برای خواندن یا نوشتن اطلاعات آدرس دهی و فعال میکند. ورودی این تابع آدرس مورد نظر است.

```
static void SRAM_RowBankActive(unsigned int ad)
{
	GPIOC->BSRR=0xC0012006;
	GPIOC->BSRR=0xC0012006;
	GPIOC->BSRR=0xC0012006;
	GPIOA->BSRR=0xFF0000;
	GPIOB->BSRR=0xFF000000;
	GPIOA->BSRR=(ad & 0xFF);
	GPIOA->BSRR=(ad & 0xFF);
	GPIOB->BSRR=(ad & 0xFF00);
	GPIOB->BSRR=(ad & 0xFF00);
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
}
```

3-4-5- تابع SRAM_Write:

این تابع عملیات نوشتن روی خانه های حافظه را انجام می دهد. ورودی این تابع شامل آدرس ستون مورد نظر(ad)، اطلاعاتی که قرار است نوشته شود (data) و حالت نوشتن (mode) می باشد. در صورتی که حالت نوشتن برابر 1 باشد،بیانگر حالت Single Mode است که عملیات نوشتن فقط در یک خانه از حافظه که آدرس دهی شده انجام می شود. اگر حالت نوشتن برابر صفر بود، بیانگر حالت BurstMode یا نوشتن پشت سر هم است که در این حالت، عملیات نوشتن در خانه ی بعد از آن خانه که قبلا درآن عملیات نوشتن صورت گرفته انجام می شود. برای استفاده از این حالت باید اول نقطه شروع را با حالت Single Mode انتخاب کنیم.

```
static void SRAM_Write(unsigned int ad,unsigned int data,char mode)
{
	if(mode==0)
```

```
	{
		HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
		HAL_GPIO_WritePin(GPIOC, RAM_CS_Pin|RAM_CAS_Pin|RAM_WE_Pin,
GPIO_PIN_RESET);
		HAL_GPIO_WritePin(GPIOC, RAM_RAS_Pin, GPIO_PIN_SET);
		GPIOA->BSRR=0xFF0000;
		GPIOB->BSRR=0xFF000000;
		GPIOA->BSRR=(ad & 0xFF);
		GPIOB->BSRR=(ad & 0xFF00)|(1<<10);
		GPIOE->MODER=0x55555555;
		GPIOE->ODR=data;
		HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_SET);
		HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
		HAL_GPIO_WritePin(GPIOC, RAM_CS_Pin , GPIO_PIN_SET);
	}
	else
	{
		HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
		GPIOE->ODR=data;
		HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_SET);
	}

}
```

3-4-6- تابع SRAM_RegisterSet:

این تابع پیکره بندی حافظه RAM را انجام می دهد.

```
static void SRAM_RegisterSet()
{
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
	HAL_GPIO_WritePin(GPIOC, RAM_CS_Pin|RAM_RAS_Pin|RAM_WE_Pin|RAM_CAS_Pin,
GPIO_PIN_RESET);
	GPIOA->BSRR=0xFF0000;
	GPIOB->BSRR=0xFF000000;
	GPIOA->BSRR=0x27;
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_SET);
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
	HAL_GPIO_WritePin(GPIOC, RAM_CS_Pin|RAM_RAS_Pin , GPIO_PIN_SET);
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_SET);
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_SET);
	HAL_GPIO_WritePin(GPIOC, RAM_CLK_Pin, GPIO_PIN_RESET);
}
```

3-4-7- تابع SRAM_BurstStop:

این تابع عملیات نوشتن پشت سرهم را متوقف می کند.

```
static void SRAM_BurstStop(void)
{
	GPIOC->BSRR=0xC0042003;
	GPIOC->BSRR=0xC0042003;
	GPIOC->BSRR=0xC0042003;
	GPIOC->BSRR=0x4000;
```

```
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x40008000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x4000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
	GPIOC->BSRR=0x40000000;
}
```

3-4-8- تابع SRAM_Init:

این تابع حافظه RAM را آماده سازی می کند.

```
static void SRAM_Init(void)
{
	HAL_GPIO_WritePin(GPIOC, RAM_CKE_Pin, GPIO_PIN_SET);
	SRAM_Precharge();
	SRAM_AutoRefresh();
	SRAM_RegisterSet();
}
```

3-4-9- تابع PicToRam:

این تابع اطلاعات یک فایل عکس را از کارت حافظه خوانده و روی حافظه RAM قرار می دهد. ورودی این تابع شامل (Sector) مکان ذخیره سازی روی حافظه RAM و (path) مسیر فایل عکس روی کارت حافظه است.

```
static void PicToRam(char sector,const TCHAR* path)
{

	UINT wbytes=0;
	BYTE buffer[960];
	WORD color=0;
	FIL MyFile;
	int ad,bank;
	ad=sector*272;
	bank=ad & 0x6000;

	if(f_open(&MyFile,path, FA_READ)==FR_OK)
	{
		number++;
		f_lseek(&MyFile,10);
```

```
		/* Load Picture ---------------------------------------- */
		for(int j=0;j<272;j++)
		{
			f_read(&MyFile,buffer,960,&wbytes);
			color=( (buffer[0] << 8) | buffer[1] );
			SRAM_RowBankActive(j+ad);
			SRAM_Write(bank,color,0);
			for(int c=1;c<480;c++)
			{
				color=( (buffer[c*2] << 8) | buffer[c*2+1] );
				SRAM_Write(bank,color,1);
			}

			SRAM_BurstStop();
			SRAM_Precharge();
		}


		f_close(&MyFile);
	}
}
```

3-4-10- تابع LCD_Display:

این تابع نمایشگر را روشن و خاموش میکند. اگر ورودی تابع صفر باشد نمایشگر خاموش، و اگر یک باشد نمایشگر روشن می شود.

```
static void LCD_Display(char state)
{
	HAL_GPIO_WritePin(GPIOA,LCD_DIS_Pin,state);
	HAL_GPIO_WritePin(GPIOA,LCD_BLK_Pin, state);
}
```

3-4-11- تابع LCD_Init:

این تابع نمایشگر را آماده سازی می کند.

```
static void LCD_Init()
{
	HAL_GPIO_WritePin(GPIOA,LCD_HSY_Pin|LCD_CLK_Pin, GPIO_PIN_SET);
	HAL_GPIO_WritePin(GPIOC,LCD_VSY_Pin|LCD_DE_Pin, GPIO_PIN_RESET);
}
```

3-4-12- تابع LCD_Hsync:

این تابع ستون های یک سطر از نمایشگر را مقدار دهی می کند.

```
static void LCD_Hsync(char de,int bank)
{

	GPIOA->BSRR=(1<<24);
	GPIOA->BSRR=(1<<26);
	GPIOA->BSRR=(1<<10);
	GPIOA->BSRR=(1<<8);
```

```
        cnth=0;
        while(cnth<45)
        {
                GPIOA->BSRR=(1<<26);
                GPIOA->BSRR=(1<<10);
                cnth++;
        }

        if(de==1)
        {
                // SRAM_Read##############
                GPIOC->BSRR=0xC0022005;
                GPIOC->BSRR=0xC0022005;
                GPIOC->BSRR=0xC0022005;
                GPIOA->BSRR=0xFF0000;
                GPIOB->BSRR=0xFF000000;
                GPIOB->BSRR=bank;
                GPIOE->MODER=0;
                GPIOC->BSRR=0x4000;
                GPIOC->BSRR=0x4000;
                GPIOC->BSRR=0x4000;
                GPIOC->BSRR=0x40008000;
                GPIOC->BSRR=0x40008000;
                GPIOC->BSRR=0x40008000;
                GPIOC->BSRR=(1<<6);

                while(cnth<525)
                {
                        GPIOA->BSRR=(1<<10);
                        GPIOC->BSRR=0x4000;
                        GPIOA->BSRR=(1<<10);
                        GPIOC->BSRR=0x4000;
                        GPIOA->BSRR=(1<<10);
                        GPIOC->BSRR=0x4000;
                        GPIOA->BSRR=(1<<26);
                        GPIOC->BSRR=0x40000000;
                        GPIOA->BSRR=(1<<26);
                        GPIOC->BSRR=0x40000000;
                        GPIOA->BSRR=(1<<26);
                        GPIOC->BSRR=0x40000000;
                        cnth++;
                }

                GPIOC->BSRR=(1<<22);

        }
        else
        {
                while(cnth<525)
                {
                        GPIOA->BSRR=(1<<26);
                        GPIOA->BSRR=(1<<10);
                        cnth++;
                }
                SRAM_Precharge();
                SRAM_AutoRefresh();
        }
}
```

3-4-13- تابع LCD_Show:

این تابع محتوای تصویری ذخیره شده روی حافظه RAM را روی نمایشگر به نمایش می گذارد. ورودی این تابع، مکان تصویر مورد نظر روی حافظه RAM می باشد.

```
static void LCD_Show(char sector)
{
	int row;
	int bank;
	row=sector*272;
	bank=row & 0x6000;
	GPIOC->BSRR=(1<<23);
	LCD_Hsync(0,bank);
	GPIOC->BSRR=(1<<7);
	cntv=0;
	while(cntv<16)
	{
		LCD_Hsync(0,bank);
		cntv++;
	}

	cntv=0;
	while(cntv<272)
	{
		SRAM_RowBankActive(cntv+row);
		LCD_Hsync(1,bank);
		cntv++;
		SRAM_BurstStop();
		SRAM_Precharge();
		SRAM_AutoRefresh();
	}
}
```

3-4-14- تابع Slide_Show:

این تابع با افکت اسلاید، تصویر فعلی نمایشگر را با تصویر بعدی جایگزین می کند . ورودی این آدرس تصویر فعلی (a) و آدرس تصویر بعدی (b) است. اگر a از b بزرگتر باشد حالت اسلاید از بالا به پایین و درغیر این صورت حالت اسلاید از پایین به بالا اجرا می شود.

```
static void Slide_Show(char a,char b)
{
	int a_row,a_bank,b_row,b_bank;

	a_row=a*272;
	a_bank=a_row & 0x6000;

	b_row=b*272;
	b_bank=b_row & 0x6000;

	for(int i=0;i<34;i++)
	{
		GPIOC->BSRR=(1<<23);
		LCD_Hsync(0,0);
		GPIOC->BSRR=(1<<7);
		cntv=0;
		while(cntv<16)
```

```
		{
			LCD_Hsync(0,0);
			cntv++;
		}

		if(a>b)
		{
			for(int m=i*8;m<272;m++)
			{

				SRAM_RowBankActive(m+a_row);
				LCD_Hsync(1,a_bank);
				SRAM_BurstStop();
				SRAM_Precharge();
				SRAM_AutoRefresh();
			}
			for(int n=0;n<i*8;n++)
			{

				SRAM_RowBankActive(n+b_row);
				LCD_Hsync(1,b_bank);
				SRAM_BurstStop();
				SRAM_Precharge();
				SRAM_AutoRefresh();
			}
		}
		else
		{
			for(int m=272-i*8;m<272;m++)
			{

				SRAM_RowBankActive(m+b_row);
				LCD_Hsync(1,b_bank);
				SRAM_BurstStop();
				SRAM_Precharge();
				SRAM_AutoRefresh();
			}
			for(int n=0;n<272-i*8;n++)
			{

				SRAM_RowBankActive(n+a_row);
				LCD_Hsync(1,a_bank);
				SRAM_BurstStop();
				SRAM_Precharge();
				SRAM_AutoRefresh();
			}
		}
	}
}
```

15-4-3- تابع SystemClock_Config:

این تابع، تنظیمات مربوط به کلاک سیستم را انجام می دهد.کد های این تابع با توجه به تنظیماتی که در مراحل ایجاد پروژه در محیط CubeMX انجام دادیم به صورت خودکار به برنامه ما اضافه شده اند.

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
```

```
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
  /** Initializes the CPU, AHB and APB busses clocks
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 8;
  RCC_OscInitStruct.PLL.PLLN = 168;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 7;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
  /** Initializes the CPU, AHB and APB busses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK)
  {
    Error_Handler();
  }
}
```

16-4-3- تابع MX_SDIO_SD_Init:

این تابع پیکره بندی رابط SDIO را برای برقراری ارتباط با کارت حافظه انجام می دهد. کد های این تابع با توجه به تنظیماتی که در مراحل ایجاد پروژه در محیط CubeMX انجام دادیم به صورت خودکار به برنامه ما اضافه شده اند.

```
static void MX_SDIO_SD_Init(void)
{

  hsd.Instance = SDIO;
  hsd.Init.ClockEdge = SDIO_CLOCK_EDGE_RISING;
  hsd.Init.ClockBypass = SDIO_CLOCK_BYPASS_DISABLE;
  hsd.Init.ClockPowerSave = SDIO_CLOCK_POWER_SAVE_DISABLE;
  hsd.Init.BusWide = SDIO_BUS_WIDE_4B;
  hsd.Init.HardwareFlowControl = SDIO_HARDWARE_FLOW_CONTROL_DISABLE;
  hsd.Init.ClockDiv = 0;
}
```

3-4-17- تابع MX_USART۱_UART_Init:

این تابع پیکره بندی رابط UART را برای برقراری ارتباط با ماژول گیرنده مادون قرمز انجام می دهد. کد های این تابع با توجه به تنظیماتی که در مراحل ایجاد پروژه در محیط CubeMX انجام دادیم به صورت خودکار به برنامه ما اضافه شده اند.

```
static void MX_USART1_UART_Init(void)
{

  huart1.Instance = USART1;
  huart1.Init.BaudRate = 9600;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }

}
```

3-4-18- تابع MX_GPIO_Init:

این تابع پیکره بندی پایه های ورودی خروجی میکروکنترلر را انجام می دهد. کد های این تابع با توجه به تنظیماتی که در مراحل ایجاد پروژه در محیط CubeMX انجام دادیم به صورت خودکار به برنامه ما اضافه شده اند.

```
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOE, GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
                          |GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|GPIO_PIN_9
                          |GPIO_PIN_10|GPIO_PIN_11|GPIO_PIN_12|GPIO_PIN_13
                          |GPIO_PIN_14|GPIO_PIN_15|GPIO_PIN_0|GPIO_PIN_1,
GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOC, RAM_CKE_Pin|RAM_CLK_Pin|RAM_CS_Pin|RAM_RAS_Pin
                          |RAM_CAS_Pin|RAM_WE_Pin|LCD_DE_Pin|LCD_VSY_Pin,
GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
```

```
  HAL_GPIO_WritePin(GPIOA, RAM_A0_Pin|RAM_A1_Pin|RAM_A2_Pin|RAM_A3_Pin
                          |RAM_A4_Pin|RAM_A5_Pin|RAM_A6_Pin|RAM_A7_Pin
                          |LCD_HSY_Pin|LCD_DIS_Pin|LCD_CLK_Pin|LCD_BLK_Pin,
GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, RAM_A10_Pin|RAM_A11_Pin|RAM_A12_Pin|RAM_A13_Pin
                          |RAM_A14_Pin|RAM_A15_Pin|RAM_A8_Pin|RAM_A9_Pin,
GPIO_PIN_RESET);

  /*Configure GPIO pins : PE2 PE3 PE4 PE5
                           PE6 PE7 PE8 PE9
                           PE10 PE11 PE12 PE13
                           PE14 PE15 PE0 PE1 */
  GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5
                          |GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|GPIO_PIN_9
                          |GPIO_PIN_10|GPIO_PIN_11|GPIO_PIN_12|GPIO_PIN_13
                          |GPIO_PIN_14|GPIO_PIN_15|GPIO_PIN_0|GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
  HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

  /*Configure GPIO pins : RAM_CKE_Pin RAM_CLK_Pin RAM_CS_Pin */
  GPIO_InitStruct.Pin = RAM_CKE_Pin|RAM_CLK_Pin|RAM_CS_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /*Configure GPIO pins : RAM_RAS_Pin RAM_CAS_Pin RAM_WE_Pin LCD_DE_Pin
                           LCD_VSY_Pin */
  GPIO_InitStruct.Pin = RAM_RAS_Pin|RAM_CAS_Pin|RAM_WE_Pin|LCD_DE_Pin
                          |LCD_VSY_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /*Configure GPIO pins : IN_TR_Pin IN_PH_Pin IN_UP_Pin */
  GPIO_InitStruct.Pin = IN_TR_Pin|IN_PH_Pin|IN_UP_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /*Configure GPIO pins : RAM_A0_Pin RAM_A1_Pin RAM_A2_Pin RAM_A3_Pin
                           RAM_A4_Pin RAM_A5_Pin RAM_A6_Pin RAM_A7_Pin
                           LCD_HSY_Pin LCD_DIS_Pin LCD_CLK_Pin */
  GPIO_InitStruct.Pin = RAM_A0_Pin|RAM_A1_Pin|RAM_A2_Pin|RAM_A3_Pin
                          |RAM_A4_Pin|RAM_A5_Pin|RAM_A6_Pin|RAM_A7_Pin
                          |LCD_HSY_Pin|LCD_DIS_Pin|LCD_CLK_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : IN_DN_Pin IN_OVL_Pin IN_DO_Pin */
  GPIO_InitStruct.Pin = IN_DN_Pin|IN_OVL_Pin|IN_DO_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

```
  /*Configure GPIO pins : RAM_A10_Pin RAM_A11_Pin RAM_A12_Pin RAM_A13_Pin
                          RAM_A14_Pin RAM_A15_Pin RAM_A8_Pin RAM_A9_Pin */
  GPIO_InitStruct.Pin = RAM_A10_Pin|RAM_A11_Pin|RAM_A12_Pin|RAM_A13_Pin
                        |RAM_A14_Pin|RAM_A15_Pin|RAM_A8_Pin|RAM_A9_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /*Configure GPIO pins : IN_A_Pin IN_B_Pin IN_C_Pin IN_D_Pin
                          IN_E_Pin IN_F_Pin IN_G_Pin SW1_Pin
                          SW2_Pin IN_10_Pin */
  GPIO_InitStruct.Pin = IN_A_Pin|IN_B_Pin|IN_C_Pin|IN_D_Pin
                        |IN_E_Pin|IN_F_Pin|IN_G_Pin|SW1_Pin
                        |SW2_Pin|IN_10_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

  /*Configure GPIO pin : IN___Pin */
  GPIO_InitStruct.Pin = IN___Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(IN___GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pin : LCD_BLK_Pin */
  GPIO_InitStruct.Pin = LCD_BLK_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LCD_BLK_GPIO_Port, &GPIO_InitStruct);
}
```

3-5- نتیجه گیری:

یکی از مهم ترین پارامتر ها در طراحی یک سیستم میکروکنترلری، برنامه نویسی آن است. با استفاده از ابزار مناسب برنامه نویسی شامل کامپایلر و دستگاه دیباگر، توانستیم برنامه ای پر سرعت و بدون خطا برای این پروژه بنویسیم. در برنامه نویسی سعی بر این بود که تا حد ممکن کد ها بهینه باشند و برنامه را به توابع و زیر کد ها شکستیم.

شکل 3-6 : پروگرامر STLINK-V2

فصل ۴– عملکرد دستگاه:

4-1- مقدمه:

برای آشنایی با نحوه راه اندازی دستگاه و خلاصه ای از نحوه عملکرد آن، این بخش را دنبال کنید.

4-2- نحوه عملکرد دستگاه:

در ابتدا و پس از روشن شدن دستگاه، فایل هایی که نام آن ها شماره ای از 1 تا 100 باشد و پسوند آن ها rgb باشد از کارت حافظه فراخوانی می شوند و در حافظه Ram قرار می گیرند. سپس میکروکنترلر برای نمایش یک عکس، اطلاعات پیکسل به پیکسل آن را از حافظه Ram فراخوانی می کند واز آن جا که خطوط دیتای ال سی دی و حافظه RAM مشترک هستند، اطلاعات هر پیکسل مستقیما از حافظه RAM روی درگاه نمایشگر قرار میگیرد و میکروکنترلر فقط عملیات آدرس دهی حافظه RAM و سیگنال دهی ال سی دی را انجام می دهد. استفاده از این روش باعث افزایش نرخ نمایش تصاویر روی صفحه نمایش و عدم نیاز بهLTDC می شود.

در حین کارکرد دستگاه هر زمان که ماژول گیرنده مادون قرمزاطلاعاتی را از ریموت کنترل دریافت کند وقفه رابط سریال اتفاق می افتد و اطلاعات دریافتی در برنامه اصلی بررسی شده و عملیات مورد نظر انجام می شود.

4-3- نحوه انتقال عکس به کارت حافظه :

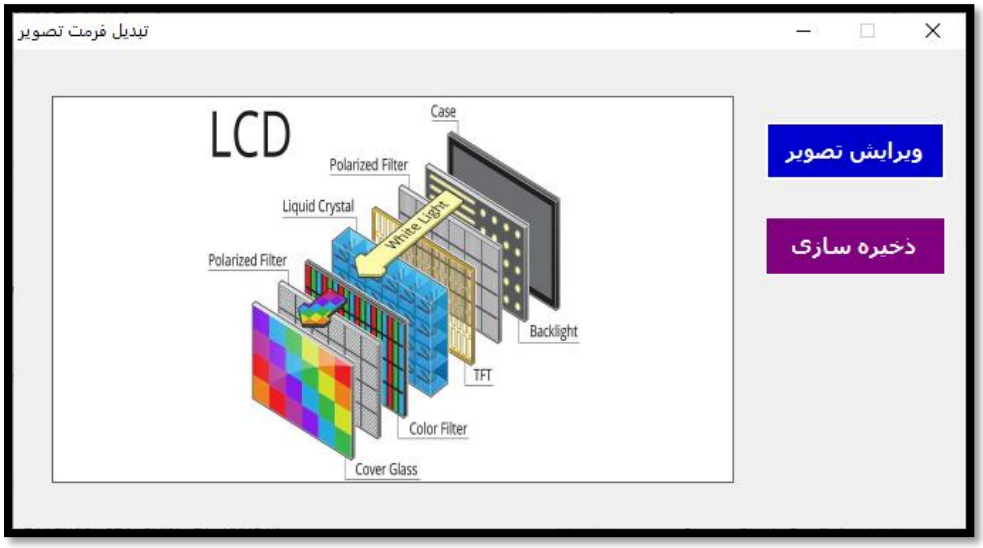
برای انتقال تصاویر به کارت حافظه دستگاه ابتدا باید تصاویر را با نرم افزار تبدیل فرمت باز کرده و با نام های 1 تا 100 روی کارت حافظه ذخیره کنیم. این نرم افزار اطلاعات تک تک پیکسل های تصویر که شامل 16 بیت RGB565 (دو بایت) و 480 در 272 پیکسل می باشد را به صورت خام و بدون فشرده سازی، به ترتیب روی یک فایل با پسوند rgb ذخیره می کند تا میکروکنترلر از دیکود کردن فایل های تصویری بی نیاز شود. با توجه به گفته های بالا حجم هر فایل rgb به صورت زیر محاسبه می شود :

480 * 272 = 130560 پیکسل

130560 * 2 = 261120 بایت

261120 / 1024 = 255 کیلو بایت

بنابر این حجم هر یک از فایل های تبدیل فرمت شده 255 کیلوبایت است. در شکل زیر محیط نرم افزار تبدیل فرمت را مشاهده می کنید. ابتدا با انتخاب گزینه ویرایش تصویر، تصویر مورد نظر را با هر نوع فرمت تصویر و اندازه ای انتخاب می کنیم. نرم افزار عکس را فراخوانی و سایز آن را تغییر می دهد و به نمایش در می آورد. سپس یا انتخاب گزینه ذخیره سازی، فایل rgb مربوطه را ذخیره می کنیم.



شکل 1-4 : محیط نرم افزار تبدیل فرمت تصویر

این نرم افزار توسط بنده تحت نرم افزار Visual Studio و به زبان #C نوشته شده است.

4-4- نتیجه گیری

برای آسان تر شدن کار و بی نیاز شدن از تغییر فرمت و اندازه ی تصویر به صورت دستی، یک نرم افزار فارسی طراحی شده که این کار را به راحتی و تنها با چند کلیک برای ما انجام می دهد.

فصل ۵– نتیجه‌گیری و پیشنهادها

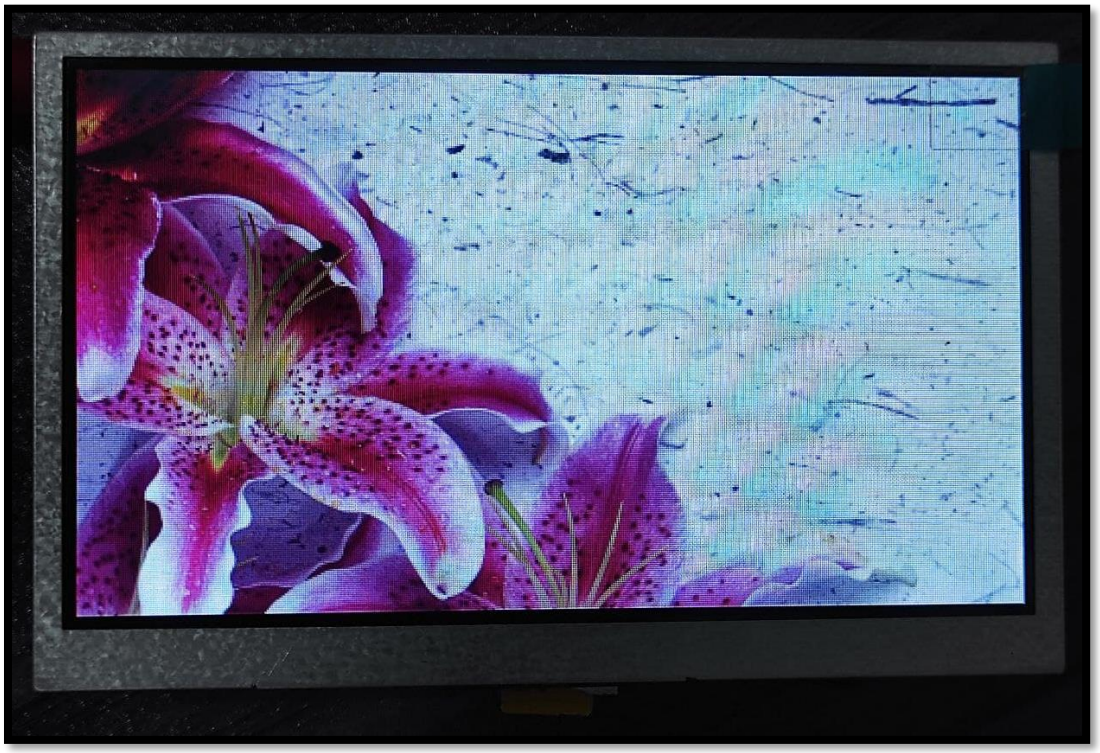
5-1- نتیجه‌گیری

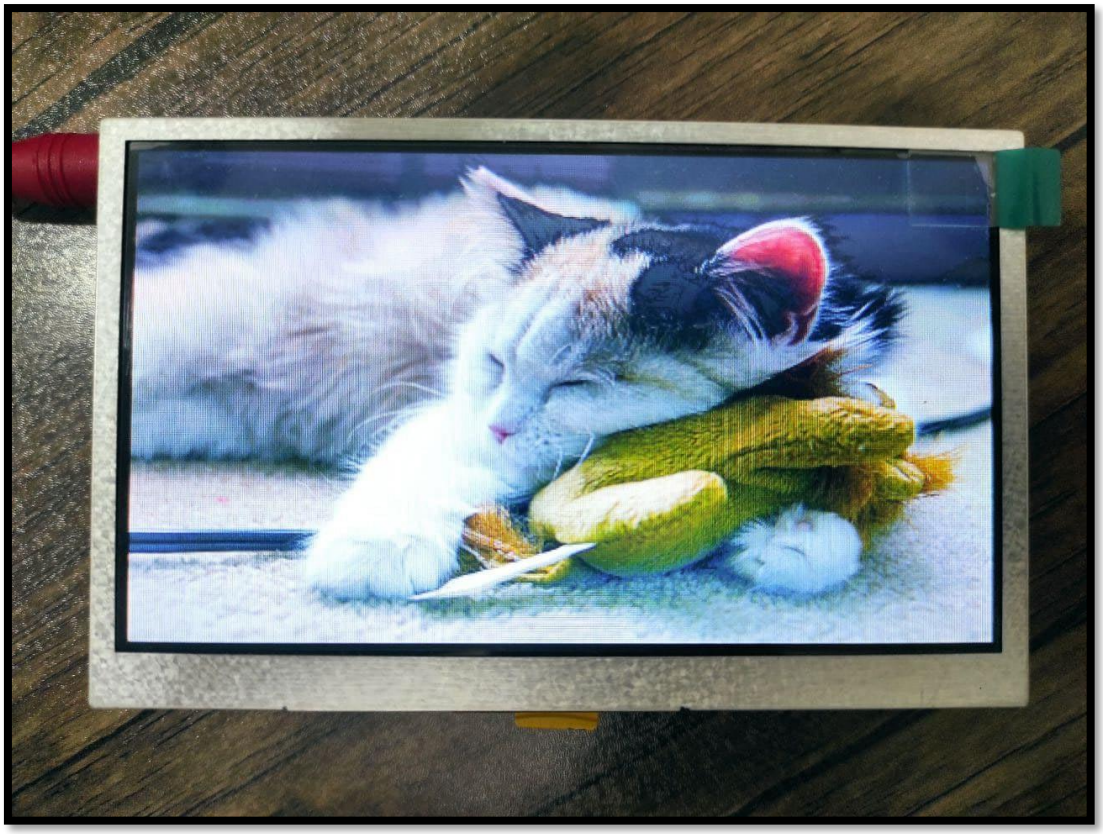
- برای ساخت یک پنل نمایشگر باید ابتدا با نحوه عملکرد نمایشگر ها آشنا شد و سپس برای آن ها یک سخت افزار مناسب طراحی کرد.
- با استفاده از بسته های نرم افزاری نظیر FatFs میتوان یک سیستم مدیریت فایل را برای دسترسی به اطلاعات ذخیره شده روی کارت حافظه داشت.
- در یک سیستم میکروکنترلری بحث برنامه نویسی و همچنین مدیریت اطلاعات بسیار حائز اهمیت است.
- برای طراحی سخت افزار می بایست برای تغذیه هر قطعه برنامه ریزی کرد و یک منبع تغذیه با خروجی پایدار و توانی بیشتر از توان قطعه برای آن در نظر گرفت.

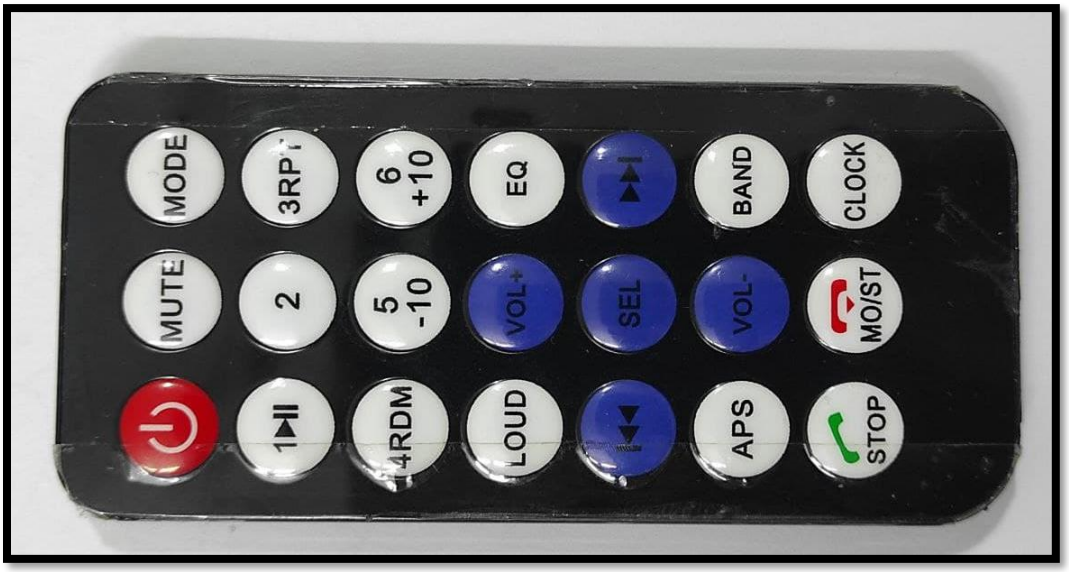
5-2- پیشنهادها

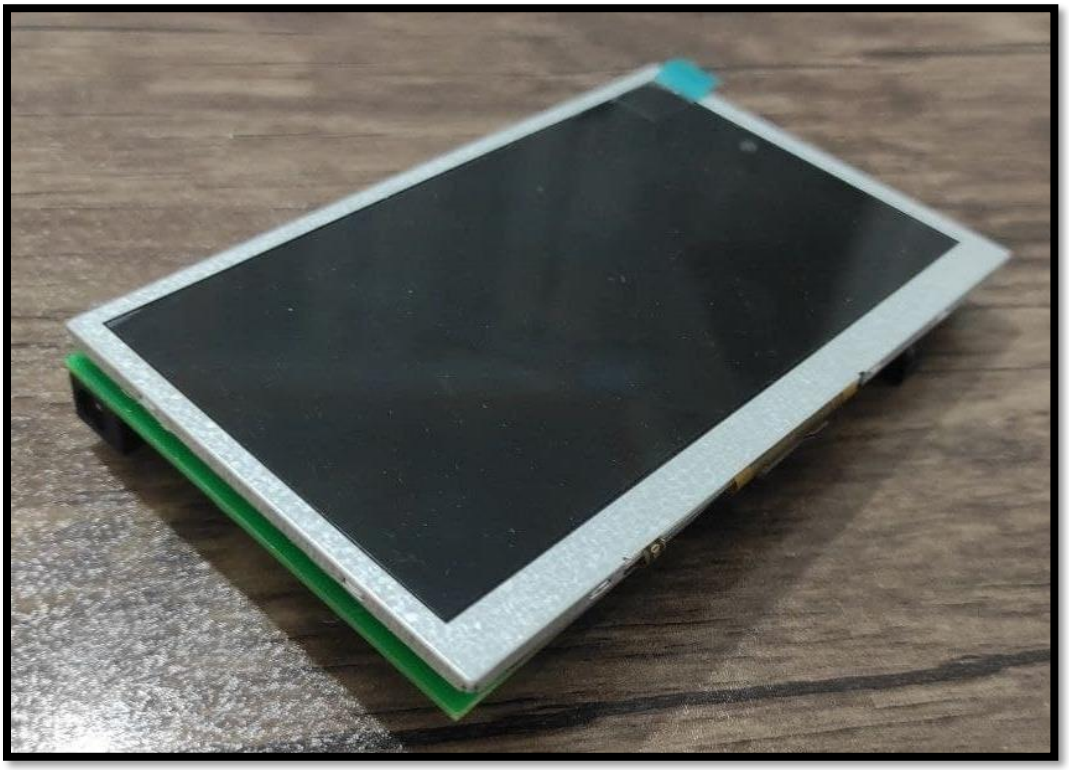
می توانستیم این پروژه را با تراشته های ساخت شرکت Allwinner نظیر تراشه F1C100S یا V3S هم انجام دهیم. این تراشه ها دارای دیکودرها و انکودر های سخت افزاری صوت و تصویر هستند که می توان به راحتی نمایشگر های مختلف را به آن ها متصل کرد. اما ازآنجا منابع آموزشی در زمینه این تراشه بسیار محدود است تصمیم به استفاده از میکروکنترلر های ARM گرفته شد.

ضمیمه أ – عکس هایی از پروژه






MODE
MUTE
3RPT
2
1
6
+10
5
-10
4RDM
EQ
VOL+
LOUD
SEL
VOL-
BAND
APS
CLOCK
MO/ST
STOP



فهرست مراجع:

1- STMicroelectronic User manual UM1721 PDF File “Developing applications on STM32Cube™ with FatFs”.

2- STMicroelectronic Reference manual RM0090 PDF File “STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm®-based 32-bit MCUs”.

3- STMicroelectronic Data brief PDF File “STM32Cube MCU Package for STM32F4 Series with HAL, low-layer drivers and dedicated middlewar”.

4- NXP PDF File “Introduction to graphics and LCD Introduction to graphics and LCD technologies”.

5- Samsung K4S561632C Datasheet PDF File.

6- Samsung SDRAM Timing Diagram PDF File.

7- Samsung SDRAM Device Operations PDF File.

8- Fairchild 2N7002 Datasheet PDF File.

9- Innolux AT050TN33 LCD Module SPECIFICATION PDF File.

10- Texas Instruments TPS6104x Datasheet PDF File.

واژه نامه‌ی فارسی به انگلیسی

Adaptor	آداپتور
Chipset	تراشه
Column	سطر
Compile	تدوین
Compiler	تدوین کننده
Crystal	کریستال
Debugger	اشکال زدا
Direct Current	جریان مستقیم
Display	نمایشگر
External Intrrupt	وقفه خارجی
Forward Current	جریان مستقیم
Frequency	فرکانس
HAL	لایه انتزاع سخت افزار
Image File	فایل تصویری
Inductor	سلف
InfraRed Remote	ریموت مادون قرمز
Interface	درگاه
LCD	نمایشگر کریستال مایع
Linear Regulator	رگولاتور خطی
Intrrupt	وقفه
Microcontroller	میکروکنترلر
PCB	مدار چاپی
Pixel	پیکسل
Resolution	دقت
Reverse Voltage	ولتاژ معکوس
Row	سطر
SD Card	کارت حافظه

واژه نامه‌ی انگلیسی به فارسی

Adaptor	آداپتور
Chipset	تراشه
Column	سطر
Compile	تدوین
Compiler	تدوین کننده
Crystal	کریستال
Debugger	اشکال زدا
Direct Current	جریان مستقیم
Display	نمایشگر
External Intrrupt	وقفه خارجی
Forward Current	جریان مستقیم
Frequency	فرکانس
HAL	لایه انتزاع سخت افزار
Image File	فایل تصویری
Inductor	سلف
InfraRed Remote	ریموت مادون قرمز
Interface	درگاه
LCD	نمایشگر کریستال مایع
Linear Regulator	رگولاتور خطی
Intrrupt	وقفه
Microcontroller	میکروکنترلر
PCB	مدار چاپی
Pixel	پیکسل
Resolution	دقت
Reverse Voltage	ولتاژ معکوس
Row	سطر
SD Card	کارت حافظه

Abstract

With the advancement of science and the advent of LCD monitors, various applications have been defined for this type of monitors in different fields. Currently, microcontrollers designing and manufacturing companies put a separate hardware section on their chipsets to set up and display Image content on such screens that named LTDC.

In this project, an attempt has been made to implement this feature on microcontrollers without LTDC by software developing.

Photo Player Panel with Remote Control

Thesis
Submitted in Partial Fulfillment of the
Requirements for the Degree of Bachelor of Science (B.Sc.)
in Electrical Engineering, [Electronics, Communication Systems, Control Systems, Power Systems]

Department of Electrical and Computer Engineering
School of Engineering
Quchan University of Technologies

By:

Mohammad Hadi Najafi

Supervisor:

Dr. Morteza Rajabzadeh

Examiner:

Dr. Maryam Zare

winter 2021